

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Core Python File Report

Hierarchical Reinforcement Learning for Opportunistic Aircraft Maintenance

A line-by-line guide to the maintenance tutorial

Soroush Asadian
Student Number: 270334

Supervisor: Emanuele Petriconi

22 October 2026

Contents

1 Purpose and scope **4**

2 How the program works **4**

2.1 What is simulated, trained, and evaluated 4

2.2 Where actions come from 5

2.3 Neural-network pieces and their roles 5

2.4 Relationship to multi-agent reinforcement learning 6

2.5 Dataclasses and decorators 7

2.6 Observation, action, and encoding conventions 7

2.7 Duration and the PPO update 8

2.8 Call and data flow 8

3 Source walkthrough **8**

4 Module prologue **9**

4.1 Module documentation, future import, and initial imports 9

5 The one-item simulator **10**

5.1 Module imports and declarations 10

5.2 Class `MiniMaintenanceEnv` 10

5.2.1 `MiniMaintenanceEnv.__init__` 11

5.2.2 `MiniMaintenanceEnv.reset` 12

5.2.3 `MiniMaintenanceEnv.step` 13

5.2.4 `MiniMaintenanceEnv._observation` 14

5.2.5 `MiniMaintenanceEnv.render_text` 15

5.3 Function `random_policy` 15

5.4 Function `threshold_policy` 15

5.5 Function `rollout` 16

5.6 Module-level declarations 16

6 The small hierarchical action example **17**

6.1 Module imports and declarations 17

6.2 Function `valid_action` 17

6.3 Function `all_actions` 19

6.4 Class `ConditionalMaintenanceEnv` 19

6.4.1 `ConditionalMaintenanceEnv.__init__` 19

6.4.2 `ConditionalMaintenanceEnv._observation` 20

6.4.3 `ConditionalMaintenanceEnv.reset` 20

6.4.4 `ConditionalMaintenanceEnv.step` 21

6.5 Module-level declarations . . . 22

7 The Gymnasium interface **22**

7.1 Module imports and declarations . . . 22

7.2 Class `GymMaintenanceEnv` . . . 23

7.2.1 `GymMaintenanceEnv.__init__` . . . 23

7.2.2 `GymMaintenanceEnv._encode` . . . 24

7.2.3 `GymMaintenanceEnv.reset` . . . 24

7.2.4 `GymMaintenanceEnv.step` . . . 25

7.2.5 `GymMaintenanceEnv.render` . . . 25

7.2.6 `GymMaintenanceEnv.close` . . . 25

7.3 Module-level declarations . . . 25

8 The fleet simulator and rule baseline **26**

8.1 Module imports and declarations . . . 26

8.2 Class `Config` . . . 27

8.2.1 `Config.__post_init__` . . . 28

8.3 Class `Plan` . . . 29

8.4 Class `Job` . . . 30

8.5 Class `FleetMaintenanceEnv` . . . 30

8.5.1 `FleetMaintenanceEnv.__init__` . . . 31

8.5.2 `FleetMaintenanceEnv.reset` . . . 32

8.5.3 `FleetMaintenanceEnv.split_observation` . . . 34

8.5.4 `FleetMaintenanceEnv.idle_action` . . . 34

8.5.5 `FleetMaintenanceEnv.resource_state` . . . 34

8.5.6 `FleetMaintenanceEnv.sensor_noise` . . . 35

8.5.7 `FleetMaintenanceEnv._sensor_model` . . . 35

8.5.8 `FleetMaintenanceEnv._clip_measurements` . . . 36

8.5.9 `FleetMaintenanceEnv._invert_sensor_metrics` . . . 36

8.5.10 `FleetMaintenanceEnv.observation_metrics` . . . 37

8.5.11 `FleetMaintenanceEnv._observation` . . . 37

8.5.12 `FleetMaintenanceEnv.validate_row` . . . 38

8.5.13 `FleetMaintenanceEnv._start_job` . . . 39

8.5.14 `FleetMaintenanceEnv._update_filter` . . . 40

8.5.15 `FleetMaintenanceEnv.step` . . . 41

8.5.16 `FleetMaintenanceEnv.render` . . . 46

8.6 Function `threshold_action` . . . 46

8.7 Module-level declarations . . . 48

9 The action-shape adapter **48**

9.1 Module imports and declarations . . . 49

9.2 Class `FlattenFleetAction` . . . 49
9.2.1 `FlattenFleetAction.__init__` . . . 49
9.2.2 `FlattenFleetAction.action` . . . 50
9.2.3 `FlattenFleetAction.reverse_action` . . . 50
9.3 Module-level declarations . . . 50

10 The actor-critic policies and option executor **51**
10.1 Module imports and declarations . . . 51
10.2 Class `FleetActorCritic` . . . 51
10.2.1 `FleetActorCritic.__init__` . . . 52
10.2.2 `FleetActorCritic.value` . . . 53
10.2.3 `FleetActorCritic.act` . . . 54
10.3 Class `OptionActorCritic` . . . 57
10.3.1 `OptionActorCritic.__init__` . . . 57
10.3.2 `OptionActorCritic.value` . . . 58
10.3.3 `OptionActorCritic.act` . . . 58
10.4 Class `LearnedOptionActorCritic` . . . 58
10.4.1 `LearnedOptionActorCritic.__init__` . . . 59
10.4.2 `LearnedOptionActorCritic.value` . . . 59
10.4.3 `LearnedOptionActorCritic.act` . . . 60
10.5 Module imports and declarations . . . 60
10.6 Function `execute_option` . . . 61
10.7 Module-level declarations . . . 62

11 Training, checkpointing, and evaluation **63**
11.1 Module imports and declarations . . . 63
11.2 Function `make_model` . . . 64
11.3 Function `load_checkpoint` . . . 64
11.4 Function `smdp_gae` . . . 65
11.5 Function `train` . . . 66
11.6 Function `evaluate` . . . 69
11.7 Function `main` . . . 70
11.8 Module-level declarations . . . 72

12 Interpretation and limitations **72**

1. Purpose and scope

This report explains the core tutorial module in source order. It uses the annotated core file as the source of the prior line-by-line notes and presents executable Python without those explanation-only comments in the Minted listings. The excerpts are embedded in this TeX file, with their original Python source line numbers preserved, so no separate code file is required to compile the report. Blank lines are omitted from the explanation lists because they only separate code visually. Docstrings, decorators, imports, declarations, expressions, and control-flow lines are explained.

The core module includes small teaching environments, the fleet simulator, action adapters, neural policies, an option executor, and training and evaluation routines. The separate parts-and-solutions file and behavioral-test file are outside this report's code scope. The simulator's parameters and hazard mechanisms are educational assumptions. The module itself states that they are not calibrated real turbofan data.

2. How the program works

2.1. What is simulated, trained, and evaluated

The simulator. `FleetMaintenanceEnv` is a software environment with hidden component health, noisy condition readings, an approximate particle filter, event and wear processes, aircraft jobs, spare inventory, crews, workshops, rewards, and episode metrics. At each step it receives an action and returns the next observation, reward, terminal flags, and diagnostic information. The simulator's rules and configuration stay fixed during learning.

The policy and value estimator. A policy maps the observation available to the agent to an action distribution. The actor chooses actions. The critic estimates $V(s)$, the expected future return from an observation. PyTorch's `nn.Module` and registered layers hold the actor and critic parameters that the optimizer updates. This is the standard actor–critic division used in reinforcement learning [1].

Training. `train()` repeatedly calls `model.act()` with no action supplied, so the model samples an action during rollout. It sends that action to `env.step()` or, for manager policies, to `execute_option()`, which advances the same environment for several days. The resulting rewards and observations form the training data. The code then estimates advantages and return targets, recomputes the probability of the stored actions under the updated policy, and takes optimizer steps. Thus training updates neural-network weights; it does not train the maintenance simulator or calibrate its physical assumptions. The clipped surrogate, value error, and entropy term make the update PPO-style [2].

The `steps` budget counts simulated days. `rollout_size` counts decisions gathered before an update, so these quantities diverge when an option lasts more than one day. For each rollout, the code calculates duration-aware advantages, normalizes them, and performs four passes over shuffled minibatches of up to 64 transitions. Adam uses a learning rate of 3×10^{-4}; the policy ratio is clipped to $[0.8, 1.2]$, the value-error term is weighted by 0.5, the entropy term by 0.001, and gradient norm is clipped at 0.5.

Evaluation. `evaluate()` runs new episodes at specified seeds. For a learned model it requests deterministic actions, so it selects the most probable action rather than sampling. For `threshold`, it calls the rule-based baseline directly. Evaluation records metrics and does not call an optimizer.

The reported availability is the simulated number of flights divided by aircraft count times horizon. Other records include the simulator's accumulated return, cost, groundings,

replacements, failures, downtime, event counts, and rejections. These metrics describe the simulated episodes under the chosen configuration.

2.2. Where actions come from

For `flat`, `conditional`, and `shared`, `FleetActorCritic.act()` produces an eight-field row per aircraft: grounding, engine mask, two component bundles, crew allocation, workshop, work-order start, and priority. It forms each categorical distribution from the observation. All three modes apply structural masks; conditional and shared modes also let earlier field choices influence later logits. The environment still performs resource checks and can reject a proposal, so policy masks and simulator acceptance are related but distinct safeguards.

For `options`, the manager samples one of four integer actions: wait one day, targeted maintenance, opportunistic bundle, or preventive bundle. `execute_option()` translates that choice into a plan, using `threshold_action()` unless the option is wait. For `learned_options`, the manager selects a macro and a goal-conditioned worker selects the fleet plan. The fixed executor then applies it. The code describes these macros as latent planning contexts; this implementation does not learn certified reusable skills or implement canonical Option-Critic. Classical options provide a framework for temporally extended decisions, and Option-Critic is a particular method for learning intra-option policies and termination rules [4, 5].

When the trainer updates PPO, it does pass the stored action to `model.act(obs, actions)`. In that call, the model does not choose a new action. It recomputes the stored action's log-probability and entropy so the loss can compare the current policy with the policy that generated the rollout.

2.3. Neural-network pieces and their roles

`nn.Linear` is a learned affine transformation. `nn.Sequential` composes layers into an encoder or multi-layer perceptron. `nn.ModuleList` registers a variable number of per-action-field heads and conditioning layers, so PyTorch includes their weights in device moves, checkpoints, and optimization. `Categorical` turns logits into a discrete distribution and supplies sampling, log-probability, and entropy.

In `FleetActorCritic`, `global_encoder` creates shared fleet context for the actor. It is distinct from `critic`, which reads the full observation and produces a single value estimate. Earlier selected action fields are also fed through the conditional projections so later action heads can depend on previous choices. In `LearnedOptionActorCritic`, the manager chooses the goal and the worker receives it as a one-hot vector.

The three modes are alternative daily fleet policies. Each produces the same eight-field action row for every aircraft: grounding, engine mask, two component bundles, crew allocation, workshop, work-order start, and priority. They differ in the information and dependencies used by the actor:

Mode	Actor context	How action fields depend on one another
`flat`	Local aircraft features, public fleet information, and a learned whole-fleet summary.	Each field's logits come from the shared aircraft representation, without feeding previous choices into the next head.
`conditional`	The same context as `flat`.	Each later head also receives a learned projection of the previously chosen fields.
`shared`	Local aircraft features and public fleet information, without the learned whole-fleet summary.	As in `conditional`, previous choices are fed into later heads.

The actor chooses fields in order. In `conditional`, for example, the grounding choice is encoded in the previous-choice vector and can influence the engine-mask distribution; that engine choice can then influence the bundle distributions. This is learned dependence: earlier choices change the later heads' logits and therefore their probabilities.

All three modes also apply structural masks to remove choices that do not fit the choices already made. Consequently, `flat` does not make every field unconditionally independent: its later logits do not learn from earlier choices, but the valid categories for later fields can still depend on them. For example, after the grounding decision, the mask can rule out engine or bundle values that would form an invalid row. These policy masks handle action structure; the simulator separately checks resource availability and may reject a proposal it cannot execute.

There is an implementation detail behind the name `shared`. A single actor and the same action heads are applied to all aircraft in all three modes, so their weights are reused across aircraft in every mode. The `shared` variant's distinctive input choice is that it omits `global_encoder`; it still uses the previous-choice conditioning layers. Each mode is trained as its own model, and its weights are not shared with the other modes. The critic is centralized in all three: it receives the full fleet observation and returns one value estimate. These are daily policies; the separate options manager chooses a high-level plan that can run for multiple days.

2.4. Relationship to multi-agent reinforcement learning

This tutorial does not implement or evaluate a multi-agent reinforcement learning (MARL) method. It trains one centralized fleet-level policy with one custom PPO loop and the simulator's single fleet-level reward signal. At each daily decision, that policy produces a joint action containing one row per aircraft; the rows' log-probabilities are combined, and one centralized critic estimates value from the full fleet observation. The actor parameters are reused across aircraft. Per-aircraft action rows and parameter sharing can also be used in MARL, but here they are features of one fleet-level controller: the code does not formulate aircraft as separate learning agents or train and evaluate a multi-agent learning algorithm. Resource feasibility and conflicts are handled centrally by the policy masks and simulator resolver, rather than by learned agent-to-agent coordination. In particular, `shared` names an actor-architecture variant that uses aircraft-local features together with public fleet information; it does not mean that independent aircraft agents learn or coordinate. Similarly, the manager and worker in `LearnedOptionActorCritic` are hierarchical decision levels inside the single fleet controller, not separate aircraft agents.

The method studied here is therefore centralized fleet-level reinforcement learning with per-aircraft action outputs, and this report makes no claim to have used or evaluated MARL. MARL is reserved as a future research direction. A future study would need to state how agents are defined, what information and actions each agent receives or controls, how the agents coordinate and are trained, and how the resulting multi-agent method is evaluated.

2.5. Dataclasses and decorators

The `@dataclass` line is a decorator: Python creates the class object, passes it to `dataclass`, and binds the returned class to the same name. The decorator supplies record methods such as an initializer and representation. `@dataclass(frozen=True)` also prevents assigning a new value to a field after construction, which suits `Config` and `Plan` as records of assumptions and accepted decisions. `Job` uses ordinary `@dataclass` because its remaining duration changes while the simulator advances. The frozen setting prevents field reassignment; it is not a general deep-freeze of arbitrary nested objects.

2.6. Observation, action, and encoding conventions

Each aircraft row contains two engine bundles. Every bundle is a six-bit component mask. For component index c, the expression `1 << c` sets that component's bit; for instance, decimal 5 is 000101_2, selecting component indices 0 and 2. The two-bit engine mask has a different job: bit 0 indicates a nonempty first-engine bundle and bit 1 indicates a nonempty second-engine bundle. Therefore `int(b0 != 0) + 2 * int(b1 != 0)` returns 0, 1, 2, or 3 according to which engines have work. The factor 2 places the second engine in bit position 1; it is not a larger priority.

The component index mapping is fan = 0, low-pressure compressor = 1, high-pressure compressor = 2, combustor = 3, high-pressure turbine = 4, and low-pressure turbine = 5. Their six-bit weights are 1, 2, 4, 8, 16, and 32. For example, `5` is 000101_2: it requests component indices 0 and 2 on one engine. If the two bundles are `[5, 8]`, then the engine mask is `1 + 2 = 3`, or 11_2, because both bundles are nonzero. The bundle values still carry the component choices; the engine mask only says which engines have work. A zero bundle means no new work on that engine. A busy aircraft is different: its existing job remains committed and the simulator ignores a replacement proposal for that aircraft.

The observation has 70 local values per aircraft and 10 public fleet values. The local row summarizes particle means, spreads and lower health quantiles, synthetic exposure, busy-job status and plan fields, sensor efficiency and temperature, and time since flight. Public values describe the day, free crews, free workshops, replenishment timing, and normalized spare stock. These features are the policy's input; the simulator's true health array and event records are not part of the policy observation.

Busy aircraft have a committed job. A newly proposed action must not replace that job. Bundle zero means no new component work for that engine. All fleet policy modes mask choices that conflict with the action structure, while `validate_row()` checks consistency and `_start_job()` checks resources and stock before commitment.

The simulator's `exposure` is a synthetic aircraft-level multiplier sampled at reset. In this code it changes modeled wear and temperature; it is not an observed flight-hour measure or an empirical probability. The observation contains estimates and sensor information. Hidden health and event diagnostics remain inside the simulator and are not actor inputs.

2.7. Duration and the PPO update

The daily policies make one decision per simulated day. A manager-level option may run for several days, so one stored decision can summarize multiple primitive transitions. `smdp_gae()` accounts for this duration by using γ^d for the next-value bootstrap and $(\gamma\lambda)^d$ for advantage propagation, where d is the option duration. A terminal transition sets the bootstrap multiplier to zero. The reward returned by `execute_option()` is already discounted over the primitive days. Semi-Markov formulations make this duration explicit [4].

The PPO minibatch update evaluates the actions stored in the rollout. The ratio $\exp(\log \pi_{new} - \log \pi_{old})$ compares new and old action probabilities. Clipping the ratio to 0.8–1.2 limits the policy objective's response to large changes in that minibatch. The critic is trained toward return targets, and the negative entropy term rewards continued exploration. This is a PPO-style implementation, not evidence that the selected policy is optimal for aircraft maintenance.

Why the fleet trainer is custom. The notebook's introductory `PPO('MlpPolicy', ...)` and `model.learn()` example trains the small `GymMaintenanceEnv`. The later fleet experiments build their own actor–critic model with `make_model()` and update it inside `train()`; the SB3 example does not initialize or update these fleet weights. This choice gives the tutorial direct control over the fleet policy's sequential action fields and structural masks, and over manager options that can last several simulated days. The custom `smdp_gae()` and rollout data keep each option's duration in the return calculation. SB3 can be extended with custom policies, but preserving these action and duration behaviors would require adapting its policy, action-distribution, and rollout interfaces; its standard `MlpPolicy` path does not supply them automatically [3]. The `FlattenFleetAction` wrapper only reshapes the action for an API; it does not add the fleet policy's conditional logic or duration-aware updates. Thus `train()` is itself the custom PPO training loop: it computes the clipped PPO objective and value loss, then Adam applies the gradients. This is a flexibility and teaching choice, rather than a limitation of PPO or SB3.

2.8. Call and data flow

Path	What happens
Daily-policy rollout	`train` calls `model.act`; its action goes to `FleetMaintenanceEnv.step`; the returned observation, reward, and value data feed the PPO update.
Manager option	`train` samples a macro; `execute_option` selects or accepts a plan, repeatedly calls `env.step`, and returns the accumulated reward, duration, and stopping information.
Learned option	The manager chooses a macro; the worker receives its one-hot goal and chooses a plan; the fixed executor carries the plan through the environment.
Evaluation	`evaluate` uses deterministic policy actions or `threshold_action`, records episode metrics, and leaves model weights unchanged.

3. Source walkthrough

The sections below follow the core module's order. Each excerpt is embedded in this report from its corresponding range in the core Python source; Minted's displayed line numbers preserve the original source numbering. Its purpose is explained first, followed by an ex-

planation for each nonblank source line. Lines inside a multi-line expression are identified separately, even when their meaning depends on the statement that began above them.

4. Module prologue

The module docstring records the tutorial's scope and synthetic assumptions. The future import postpones annotation evaluation, allowing type annotations to refer to names that are defined later. The early imports and `Observation` alias support the small examples.

4.1. Module documentation, future import, and initial imports

This block establishes the module's imports, constants, or public interface.

```
# These notes are comments only; the uploaded source remains unchanged.
"""One supporting module for the comprehensive RL/HRL maintenance course.

OBSERVATION_VERSION=2 includes synthetic efficiency, temperature and reading age.
The first examples and user's exercise answers are namespaced by tutorial part.
The fleet classes/functions keep their public names. No import depends on Code.
All figures are teaching assumptions rather than real turbofan calibration.
"""
from __future__ import annotations



# ======================================================================
```

Line-by-line explanation.

1 This source comment labels or clarifies the section; Python does not execute it.

2 This docstring states module's purpose/contract: One supporting module for the comprehensive RL/HRL maintenance course. It is documentation text, not an executable operation.

4 This docstring line states: OBSERVATION_VERSION=2 includes synthetic efficiency, temperature and reading age.

5 This docstring line states: The first examples and user's exercise answers are namespaced by tutorial part.

6 This docstring line states: The fleet classes/functions keep their public names. No import depends on Code.

7 This docstring line states: All figures are teaching assumptions rather than real turbofan calibration.

8 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

9 Postpone annotation evaluation so forward references can name classes defined later without runtime lookup.

13 This comment draws a visual boundary between source sections. It has no runtime effect.

5. The one-item simulator

This first teaching model isolates the interaction loop. MiniMaintenanceEnv is the environment: it owns health and time, applies wear, and returns reward and observations. The policies are separate decision rules. rollout connects them by resetting the environment, asking a policy for an action, and passing that action to step().

The toy simulator makes cause and effect visible: action 0 operates for one day, while action 1 grounds the aircraft and replaces the item. The reward is generated by the environment, not by the policy. random_policy and threshold_policy are alternative action selectors; rollout is the shared loop that evaluates either selector.

5.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
14  # mini_maintenance.py
15  # ========================================================================
16
17
18  import random
19  from typing import TypeAlias
20  Observation: TypeAlias = tuple[int, int]
21
```

Line-by-line explanation.

14 This source comment labels or clarifies the section; Python does not execute it.

15 This comment draws a visual boundary between source sections. It has no runtime effect.

18 Use Python's pseudo-random generator for the small simulator and random policy.

19 Import the marker used to declare a type alias explicitly.

20 Declare Observation as a type-level nickname for a two-integer tuple; runtime values are ordinary tuples.

5.2. Class `MiniMaintenanceEnv`

This class a tiny one-item simulator for state, action, reward, and episode mechanics.

```
22  class MiniMaintenanceEnv:
23      """One aircraft with one replaceable item and two high-level actions.
24
25      Actions:
26          0 = operate for one day
27          1 = ground for one day and replace the item
28
29      Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.
30      A successful operation earns +5. If operation ends at health 0, it earns
31      -12 instead. Grounding/replacement earns -4 and restores health to 3.
32      Operating wear is 1, plus one extra unit with probability shock_prob.
33
34      The episode ends after ``horizon`` days. Failure itself does not end it:
35      the policy can ground and restore the item on a later day.
36      """
37      OPERATE = 0
38      GROUND_AND_REPLACE = 1
39
```

Line-by-line explanation.

22 Define class MiniMaintenanceEnv: a tiny one-item simulator for state, action, reward, and episode mechanics.

23 This docstring states MiniMaintenanceEnv's purpose/contract: One aircraft with one replaceable item and two high-level actions. It is documentation text, not an executable operation.

25 This docstring line states: Actions:

26 This docstring line states: 0 = operate for one day

27 This docstring line states: 1 = ground for one day and replace the item

29 This docstring line states: Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.

30 This docstring line states: A successful operation earns +5. If operation ends at health 0, it earns

31 This docstring line states: -12 instead. Grounding/replacement earns -4 and restores health to 3.

32 This docstring line states: Operating wear is 1, plus one extra unit with probability shock_prob.

34 This docstring line states: The episode ends after "horizon" days. Failure itself does not end it:

35 This docstring line states: the policy can ground and restore the item on a later day.

36 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

37 Define shared module constant OPERATE; other examples reuse this value.

38 Define shared module constant GROUND_AND_REPLACE; other examples reuse this value.

5.2.1 `MiniMaintenanceEnv.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
40     def __init__(self, horizon: int=12, shock_prob: float=0.0) -> None:
41         if not isinstance(horizon, int) or horizon <= 0:
42             raise ValueError('horizon must be a positive integer')
43         if not 0.0 <= shock_prob <= 1.0:
44             raise ValueError('shock_prob must be between 0 and 1')
45         self.horizon = horizon
46         self.shock_prob = float(shock_prob)
47         self._rng = random.Random()
48         self.health = 3
49         self.day = 0
50         self._needs_reset = True
```

Line-by-line explanation.

40 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

41 Validate the input invariant not isinstance(horizon, int) or horizon <= 0; the guard rejects invalid values before the state-changing code runs.

42 Raise ValueError('horizon must be a positive integer'); invalid input stops here instead of creating inconsistent state.

43 When not 0.0 <= shock_prob <= 1.0 is true, enter this block; otherwise continue to the next case.

44 Raise ValueError('shock_prob must be between 0 and 1'); invalid input stops here instead of creating inconsistent state.

45 Store this value as self.horizon on the object so later method calls can read it.

46 Store this value as self.shock_prob on the object so later method calls can read it.

47 Store this value as self._rng on the object so later method calls can read it.

48 Update self.health. it holds health values held by this toy state or simulator for later transitions.

49 Store this value as self.day on the object so later method calls can read it.

50 Store this value as self._needs_reset on the object so later method calls can read it.

5.2.2 `MiniMaintenanceEnv.reset`

rollout and the Gymnasium wrapper call reset at episode boundaries. It resets state and, when a seed is given, the environment's pseudo-random stream.

```
52     def reset(self, *, seed: int | None=None, initial_health: int=3):
53         """Start an episode and return (observation, info).
54 
55         A seed makes subsequent stochastic wear reproducible. Calling reset
56         without a seed keeps the current generator stream.
57         """
58         if initial_health not in (0, 1, 2, 3):
59             raise ValueError('initial_health must be 0, 1, 2, or 3')
60         if seed is not None:
61             self._rng.seed(seed)
62         self.health = initial_health
63         self.day = 0
64         self._needs_reset = False
65         return (self._observation(), {'day': self.day})
```

Line-by-line explanation.

52 Define reset with the shown parameters/defaults; its purpose is to start an episode and initialize its random streams and state.

53 This docstring states reset's purpose/contract: Start an episode and return (observation, info). It is documentation text, not an executable operation.

55 This docstring line states: A seed makes subsequent stochastic wear reproducible. Calling reset

56 This docstring line states: without a seed keeps the current generator stream.

57 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

58 When initial_health not in (0, 1, 2, 3) is true, enter this block; otherwise continue to the next case.

59 Raise ValueError('initial_health must be 0, 1, 2, or 3'); invalid input stops here instead of creating inconsistent state.

60 When seed is not None is true, enter this block; otherwise continue to the next case.

61 Set a pseudo-random generator's state so later stochastic outcomes can be reproduced.

62 Update self.health. it holds health values held by this toy state or simulator for later transitions.

63 Store this value as self.day on the object so later method calls can read it.

64 Store this value as self._needs_reset on the object so later method calls can read it.

65 Return the current observation, reward, terminal flags, and info in the API's documented order.

5.2.3 `MiniMaintenanceEnv.step`

The policies do not change the transition rules. They return an action; step interprets it, advances the hidden environment state, computes reward, and returns the observation that the policy can see next.

```
67      def step(self, action: int):
68          """Return (next_observation, reward, terminated, truncated, info)."""
69          if self._needs_reset:
70              raise RuntimeError('call reset() before step(), including after an episode')
71          if action not in (self.OPERATE, self.GROUND_AND_REPLACE):
72              raise ValueError('action must be 0 (operate) or 1 (ground/replace)')
73          previous_health = self.health
74          if action == self.OPERATE:
75              extra_wear = int(self._rng.random() < self.shock_prob)
76              self.health = max(0, self.health - 1 - extra_wear)
77              failed = self.health == 0
78              reward = -12.0 if failed else 5.0
79              info = {'event': 'failed_operation' if failed else 'successful_operation', 'wear':
                ↪  previous_health - self.health, 'failed': failed, 'grounded': False}
80          else:
81              self.health = 3
82              reward = -4.0
83              info = {'event': 'grounded_and_replaced', 'wear': 0, 'failed': False, 'grounded': True}
84          self.day += 1
85          terminated = self.day >= self.horizon
86          truncated = False
87          self._needs_reset = terminated
88          info['day'] = self.day
89          return (self._observation(), reward, terminated, truncated, info)
```

Line-by-line explanation.

67 Define step with the shown parameters/defaults; its purpose is to validate one action, advance the simulation, and return the transition.

68 This docstring states step's purpose/contract: Return (next_observation, reward, terminated, truncated, info). It is documentation text, not an executable operation.

69 When self._needs_reset is true, enter this block; otherwise continue to the next case.

70 Raise RuntimeError('call reset() before step(), including after an episode'); invalid input stops here instead of creating inconsistent state.

71 When action not in (self.OPERATE, self.GROUND_AND_REPLACE) is true, enter this block; otherwise continue to the next case.

72 Raise ValueError('action must be 0 (operate) or 1 (ground/replace)'); invalid input stops here instead of creating inconsistent state.

73 Store the result in previous_health so later lines can use it.

74 When action == self.OPERATE is true, enter this block; otherwise continue to the next case.

75 Store the result in extra_wear so later lines can use it.

76 Update self.health. it holds health values held by this toy state or simulator for later transitions.

77 Record whether wear left the item at health zero; this Boolean selects the failure penalty and event label.

78 Give an operation a -12 failure penalty if health reached zero; otherwise the successful flight earns +5.

79 Store the result in info so later lines can use it.

80 Otherwise, follow this alternative branch.

81 Update self.health. it holds health values held by this toy state or simulator for later transitions.

82 Calculate this step's reward and store it for the transition result and episode return.

83 Store the result in info so later lines can use it.

84 Add 1 to the running total in self.day.

85 Mark the episode terminal once the number of simulated days reaches the configured horizon.

86 Compute whether an external time limit stopped the episode and store it in truncated; later checks/updates use this named intermediate.

87 Store this value as self._needs_reset on the object so later method calls can read it.

88 Store the result in info['day'] so later lines can use it.

89 Return the current observation, reward, terminal flags, and info in the API's documented order.

5.2.4 `MiniMaintenanceEnv._observation`

This method pack internal state into the public observation layout.

```
91      def _observation(self) -> Observation:
92          return (self.health, self.horizon - self.day)
```

Line-by-line explanation.

91 Define _observation with the shown parameters/defaults; its purpose is to pack internal state into the public observation layout.

92 Return these values together; callers unpack them in the order shown.

5.2.5 `MiniMaintenanceEnv.render_text`

Small human-readable display; useful for tracing a policy.

```
94      def render_text(self) -> str:
95          """Small human-readable display; useful for tracing a policy."""
96          return f'day={self.day:2d}  health={self.health}  steps_left={self.horizon - self.day}'
```

Line-by-line explanation.

94 Define render_text with the shown parameters/defaults; its purpose is to format the toy environment state as readable text.

95 This docstring states render_text's purpose/contract: Small human-readable display; useful for tracing a policy. It is documentation text, not an executable operation.

96 Return a readable summary of the current day, health, remaining time, jobs, resources, and stock.

5.3. Function `random_policy`

Random baseline, independent of health.

```
98  def random_policy(observation: Observation, *, rng: random.Random) -> int:
99      """Random baseline, independent of health."""
100     return rng.randrange(2)
```

Line-by-line explanation.

98 Define random_policy with the shown parameters/defaults; its purpose is to sample either toy action without using health, providing a random baseline.

99 This docstring states random_policy's purpose/contract: Random baseline, independent of health. It is documentation text, not an executable operation.

100 Return rng.randrange(2) to the caller; this ends the function and supplies its result.

5.4. Function `threshold_policy`

Simple rule baseline: replace at health 0 or 1.

```
102 def threshold_policy(observation: Observation) -> int:
103     """Simple rule baseline: replace at health 0 or 1."""
104     health, _steps_left = observation
105     return MiniMaintenanceEnv.GROUND_AND_REPLACE if health <= 1 else MiniMaintenanceEnv.OPERATE
```

Line-by-line explanation.

102 Define threshold_policy with the shown parameters/defaults; its purpose is to replace at low health and operate otherwise, providing a transparent rule baseline.

103 This docstring states threshold_policy's purpose/contract: Simple rule baseline: replace at health 0 or 1. It is documentation text, not an executable operation.

104 Read health from the observation and intentionally ignore the remaining-day field when this policy only needs health.

105 Return the value produced by this expression to the caller.

5.5. Function `rollout`

This function is the interaction loop for the small example. It calls reset(), then repeatedly calls the policy and env.step(), so it is the point where a decision rule is evaluated against the simulator.

```
107 def rollout(env: MiniMaintenanceEnv, policy, *, seed: int=0):
108     """Execute one episode; policy receives observation and returns an action."""
109     observation, _info = env.reset(seed=seed)
110     rows: list[dict] = []
111     while True:
112         action = policy(observation)
113         next_observation, reward, terminated, truncated, info = env.step(action)
114         rows.append({'observation': observation, 'action': action, 'reward': reward,
        ↪  'next_observation': next_observation, 'event': info['event'], 'terminated': terminated,
        ↪  'truncated': truncated})
115         observation = next_observation
116         if terminated or truncated:
117             break
118     return rows
```

Line-by-line explanation.

107 Define rollout with the shown parameters/defaults; its purpose is to run a policy to episode end and record each state, action, reward, and event.

108 This docstring states rollout's purpose/contract: Execute one episode; policy receives observation and returns an action. It is documentation text, not an executable operation.

109 Unpack the observation and info from reset(); assign info to _ because this caller does not use it.

110 Annotate rows as list[dict] to document the expected kind of value for readers and type checkers.

111 Continue the loop until a stopping condition triggers break or return.

112 Store the proposed action in action for the following checks and updates.

113 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

114 Add this item to rows so it stays available for later use.

115 Compute the state visible to the policy and store it in observation; later checks/updates use this named intermediate.

116 Check the episode/option stop condition terminated or truncated; when true, stop collecting transitions for this segment.

117 Stop this loop because its exit condition is satisfied.

118 Return rows to the caller; this ends the function and supplies its result.

5.6. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
119
120
121 # ====================================================================
```

Line-by-line explanation.

121 This comment draws a visual boundary between source sections. It has no runtime effect.

6. The small hierarchical action example

This section introduces dependent action fields with a deliberately small system. The legal component choices depend on which engine is selected. valid_action checks that relationship, all_actions enumerates the feasible set, and ConditionalMaintenanceEnv demonstrates how a structured action can be selected and applied.

The bit fields are compact encodings, not separate neural networks. A component mask records which components are requested, while an engine mask records which engines have nonempty work. The environment validates consistency before applying the action. This introduces conditional feasibility before the larger fleet action space.

6.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
122 # hierarchical_maintenance.py
123 # ======================================================================
124
125
126 from itertools import product
127 TOY_COMPONENTS = ('fan', 'compressor')
128
```

Line-by-line explanation.

122 This source comment labels or clarifies the section; Python does not execute it.

123 This comment draws a visual boundary between source sections. It has no runtime effect.

126 Generate combinations of candidate engine and component masks.

127 Define shared module constant TOY_COMPONENTS; other examples reuse this value.

6.2. Function `valid_action`

Action is (ground, engine_mask, component_mask).

```
129 def valid_action(action: tuple[int, int, int]) -> bool:
130     """Action is (ground, engine_mask, component_mask).
131
132     Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1
133     represent engine 0; bits 2-3 represent engine 1. A selected engine must
134     contain at least one selected component, and no unselected engine may
135     receive component work.
136     """
137     if len(action) != 3 or any((type(x) is not int for x in action)):
138         return False
139     ground, engines, components = action
140     if ground == 0:
141         return engines == 0 and components == 0
142     if ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15):
```

```
143             return False
144         selected_engines = sum((1 << engine for engine in range(2) if components & 3 << 2 * engine))
145         return engines == selected_engines
```

Line-by-line explanation.

129 Define valid_action with the shown parameters/defaults; its purpose is to check that grounding, engine selection, and component selection agree structurally.

130 This docstring states valid_action's purpose/contract: Action is (ground, engine_ mask, component_mask). It is documentation text, not an executable operation.

132 This docstring line states: Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1

133 This docstring line states: represent engine 0; bits 2-3 represent engine 1. A selected engine must

134 This docstring line states: contain at least one selected component, and no unselected engine may

135 This docstring line states: receive component work.

136 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

137 When len(action) != 3 or any((type(x) is not int for x in action)) is true, enter this block; otherwise continue to the next case.

138 Return False to reject the candidate action/input without changing simulator state.

139 Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

140 When grounding is off, only zero engine and component masks are legal; this all-zero tuple is the toy operate action.

141 Return engines == 0 and components == 0 to the caller; this ends the function and supplies its result.

142 Validate the input invariant ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15); the guard rejects invalid values before the state-changing code runs.

143 Return False to reject the candidate action/input without changing simulator state.

144 This toy action has one four-bit component mask: bits 0–1 belong to engine 0 and bits 2–3 to engine 1. Decimal 3 is 0011_2; shifting it by 2*engine creates that engine's pair mask. Shift binds more tightly than &: components & 3 « 2*engine means components & (3 « (2*engine)). The AND asks whether that pair contains any selected component; 1 « engine contributes engine flag 1 or 2. Summing distinct flags reconstructs the separate two-bit engine mask; it does not store component identities.

145 Accept only when the stated engine flags exactly match the engines with at least one selected component; this catches missing and extra flags.

6.3. Function `all_actions`

List the 16 feasible composite actions, including operate.

```
147 def all_actions():
148     """List the 16 feasible composite actions, including operate."""
149     return [(0, 0, 0)] + [(1, engines, components) for engines, components in product(range(1, 4),
        ↪  range(1, 16)) if valid_action((1, engines, components))]
```

Line-by-line explanation.

147 Define all_actions with the shown parameters/defaults; its purpose is to enumerate feasible toy actions so a policy can use an integer ID instead of a tuple.

148 This docstring states all_actions's purpose/contract: List the 16 feasible composite actions, including operate. It is documentation text, not an executable operation.

149 Return the value produced by this expression to the caller.

6.4. Class `ConditionalMaintenanceEnv`

This class a deterministic two-engine example demonstrating hierarchical actions and bitmasks.

```
151 class ConditionalMaintenanceEnv:
152     """Tiny deterministic simulator for one aircraft with two engines."""
153
```

Line-by-line explanation.

151 Define class ConditionalMaintenanceEnv: a deterministic two-engine example demonstrating hierarchical actions and bitmasks.

152 This docstring states ConditionalMaintenanceEnv's purpose/contract: Tiny deterministic simulator for one aircraft with two engines. It is documentation text, not an executable operation.

6.4.1 `ConditionalMaintenanceEnv.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
154     def __init__(self, horizon: int=6):
155         if not isinstance(horizon, int) or horizon <= 0:
156             raise ValueError('horizon must be positive')
157         self.horizon = horizon
158         self.health = [3, 3, 3, 3]
159         self.day = 0
160         self._needs_reset = True
```

Line-by-line explanation.

154 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

155 Validate the input invariant not isinstance(horizon, int) or horizon <= 0; the guard rejects invalid values before the state-changing code runs.

156 Raise ValueError('horizon must be positive'); invalid input stops here instead of creating inconsistent state.

157 Store this value as self.horizon on the object so later method calls can read it.

158 Update self.health. it holds health values held by this toy state or simulator for later transitions.

159 Store this value as self.day on the object so later method calls can read it.

160 Store this value as self._needs_reset on the object so later method calls can read it.

6.4.2 `ConditionalMaintenanceEnv._observation`

This method pack internal state into the public observation layout.

```
162     def _observation(self):
163         return tuple(self.health) + (self.horizon - self.day,)
```

Line-by-line explanation.

162 Define _observation with the shown parameters/defaults; its purpose is to pack internal state into the public observation layout.

163 Return the value produced by this expression to the caller.

6.4.3 `ConditionalMaintenanceEnv.reset`

This method start an episode and initialize its random streams and state.

```
165     def reset(self, initial_health=(3, 3, 3, 3)):
166         if len(initial_health) != 4 or any((h not in range(4) for h in initial_health)):
167             raise ValueError('provide four health values in 0..3')
168         self.health = list(initial_health)
169         self.day = 0
170         self._needs_reset = False
171         return (self._observation(), {})
```

Line-by-line explanation.

165 Define reset with the shown parameters/defaults; its purpose is to start an episode and initialize its random streams and state.

166 When len(initial_health) != 4 or any((h not in range(4) for h in initial_health)) is true, enter this block; otherwise continue to the next case.

167 Raise ValueError('provide four health values in 0..3'); invalid input stops here instead of creating inconsistent state.

168 Update self.health. it holds health values held by this toy state or simulator for later transitions.

169 Store this value as self.day on the object so later method calls can read it.

170 Store this value as self._needs_reset on the object so later method calls can read it.

171 Return the current observation, reward, terminal flags, and info in the API's documented order.

6.4.4 `ConditionalMaintenanceEnv.step`

This method validate one action, advance the simulation, and return the transition.

```
173     def step(self, action):
174         if self._needs_reset:
175             raise RuntimeError('reset before stepping')
176         if not valid_action(action):
177             raise ValueError(f'infeasible hierarchical action: {action}')
178         ground, engines, components = action
179         if ground:
180             for index in range(4):
181                 if components & 1 << index:
182                     self.health[index] = 3
183             n_engines = engines.bit_count()
184             n_components = components.bit_count()
185             reward = -3.0 - 2.0 * n_engines - 1.0 * n_components
186             event = 'grounded_and_replaced'
187         else:
188             self.health = [max(0, h - 1) for h in self.health]
189             reward = -15.0 if 0 in self.health else 8.0
190             event = 'failed_operation' if reward < 0 else 'successful_operation'
191         self.day += 1
192         terminated = self.day >= self.horizon
193         self._needs_reset = terminated
194         return (self._observation(), reward, terminated, False, {'event': event})
```

Line-by-line explanation.

173 Define step with the shown parameters/defaults; its purpose is to validate one action, advance the simulation, and return the transition.

174 When self._needs_reset is true, enter this block; otherwise continue to the next case.

175 Raise RuntimeError('reset before stepping'); invalid input stops here instead of creating inconsistent state.

176 When not valid_action(action) is true, enter this block; otherwise continue to the next case.

177 Raise ValueError(f'infeasible hierarchical action: {action}'); invalid input stops here instead of creating inconsistent state.

178 Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

179 When ground is true, enter this block; otherwise continue to the next case.

180 Loop through range(4), handling each index once.

181 Test component bit index: 1 « index creates a mask containing only that bit, and AND is nonzero exactly when the component was selected.

182 Update self.health[index]. it holds health values held by this toy state or simulator for later transitions.

183 Store the result in n_engines so later lines can use it.

184 Store the result in n_components so later lines can use it.

185 Charge a base grounding cost of 3, plus 2 per selected engine and 1 per replaced component; larger work bundles cost more.

186 Store the result in event so later lines can use it.

187 Otherwise, follow this alternative branch.

188 Update self.health. it holds health values held by this toy state or simulator for later transitions.

189 Calculate this step's reward and store it for the transition result and episode return.

190 Store the result in event so later lines can use it.

191 Add 1 to the running total in self.day.

192 Mark the episode terminal once the number of simulated days reaches the configured horizon.

193 Store this value as self._needs_reset on the object so later method calls can read it.

194 Return the current observation, reward, terminal flags, and info in the API's documented order.

6.5. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
195
196
197 # ======================================================================
```

Line-by-line explanation.

197 This comment draws a visual boundary between source sections. It has no runtime effect.

7. The Gymnasium interface

GymMaintenanceEnv adapts the small simulator to the reset/step interface expected by Gymnasium-based algorithms. It delegates the physical transition to MiniMaintenanceEnv and converts the observation to normalized float features. This is an API adapter, not a second maintenance model.

Gymnasium standardizes the interface between an environment and learning code. The wrapper's reset returns an observation and info; step returns next observation, reward, termination flags, and info. The normalizer maps health and remaining time to values from zero to one, which is convenient for a neural-network input.

7.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
198 # gym_maintenance.py
199 # ======================================================================
200
201
202 import gymnasium as gym
203 import numpy as np
204 from gymnasium import spaces
205
```

Line-by-line explanation.

198 This source comment labels or clarifies the section; Python does not execute it.

199 This comment draws a visual boundary between source sections. It has no runtime effect.

202 Use Gymnasium environment/wrapper base classes under the conventional alias.

203 Use NumPy for arrays, vectorized simulation, random sampling, and numeric calculations.

204 Declare and validate legal observation and action shapes.

7.2. Class `GymMaintenanceEnv`

This class a Gymnasium-compatible wrapper around the small maintenance example.

```
206 class GymMaintenanceEnv(gym.Env):
207     """Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces.
208
209     The observation is [health / 3, days_left / horizon]. Normalization keeps
210     both input features between zero and one for a neural-network policy.
211     """
212     metadata = {'render_modes': ['ansi']}
213
```

Line-by-line explanation.

206 Define class GymMaintenanceEnv: a Gymnasium-compatible wrapper around the small maintenance example.

207 This docstring states GymMaintenanceEnv's purpose/contract: Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces. It is documentation text, not an executable operation.

209 This docstring line states: The observation is [health / 3, days_left / horizon]. Normalization keeps

210 This docstring line states: both input features between zero and one for a neural-network policy.

211 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

212 Store the result in metadata so later lines can use it.

7.2.1 `GymMaintenanceEnv.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
214     def __init__(self, horizon: int=12, shock_prob: float=0.0):
215         super().__init__()
216         self.core = MiniMaintenanceEnv(horizon=horizon, shock_prob=shock_prob)
217         self.action_space = spaces.Discrete(2)
218         self.observation_space = spaces.Box(low=np.zeros(2, dtype=np.float32), high=np.ones(2,
        ↪  dtype=np.float32), dtype=np.float32)
```

Line-by-line explanation.

214 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

215 Initialize the parent class before setting subclass-specific state.

216 Store this value as self.core on the object so later method calls can read it.

217 Store this value as self.action_space on the object so later method calls can read it.

218 Store this value as self.observation_space on the object so later method calls can read it.

7.2.2 GymMaintenanceEnv._encode

reset() and step() call this to normalize health and remaining time into the observation vector passed to learning code.

```
220     def _encode(self, observation):
221         health, days_left = observation
222         return np.array([health / 3.0, days_left / self.core.horizon], dtype=np.float32)
```

Line-by-line explanation.

220 Define _encode with the shown parameters/defaults; its purpose is to perform this encode operation.

221 Read health from the observation and intentionally ignore the remaining-day field when this policy only needs health.

222 Return the value produced by this expression to the caller.

7.2.3 GymMaintenanceEnv.reset

This delegates state initialization to MiniMaintenanceEnv.reset and then encodes its observation for Gymnasium.

```
224     def reset(self, *, seed=None, options=None):
225         super().reset(seed=seed)
226         initial_health = 3 if options is None else options.get('initial_health', 3)
227         observation, info = self.core.reset(seed=seed, initial_health=initial_health)
228         return (self._encode(observation), info)
```

Line-by-line explanation.

224 Define reset with the shown parameters/defaults; its purpose is to start an episode and initialize its random streams and state.

225 Store the result in super().reset(seed so later lines can use it.

226 Store the result in initial_health so later lines can use it.

227 Unpack the observation and info from reset(); assign info to _ because this caller does not use it.

228 Return these values together; callers unpack them in the order shown.

7.2.4 GymMaintenanceEnv.step

This delegates the transition to MiniMaintenanceEnv.step and adapts its result to Gymnasium's five-item return convention.

```
230     def step(self, action):
231         observation, reward, terminated, truncated, info = self.core.step(int(action))
232         return (self._encode(observation), float(reward), terminated, truncated, info)
```

Line-by-line explanation.

230 Define step with the shown parameters/defaults; its purpose is to validate one action, advance the simulation, and return the transition.

231 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

232 Return these values together; callers unpack them in the order shown.

7.2.5 GymMaintenanceEnv.render

This method format simulator state for a human.

```
234     def render(self):
235         return self.core.render_text()
```

Line-by-line explanation.

234 Define render with the shown parameters/defaults; its purpose is to format simulator state for a human.

235 Return self.core.render_text() to the caller; this ends the function and supplies its result.

7.2.6 GymMaintenanceEnv.close

This method perform this close operation.

```
237     def close(self):
238         pass
```

Line-by-line explanation.

237 Define close with the shown parameters/defaults; its purpose is to perform this close operation.

238 Leave this block intentionally empty; pass is only a syntactic placeholder.

7.3. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
239
240
241 # ================================================================
```

Line-by-line explanation.

241 This comment draws a visual boundary between source sections. It has no runtime effect.

8. The fleet simulator and rule baseline

This is the main environment. It simulates several aircraft, two engines per aircraft, six components per engine, noisy measurements, belief particles, maintenance jobs, spare stock, crews, workshops, weather, bird and hail events, rewards, and episode metrics. The policy receives observations; the environment retains the hidden health and event tapes. Config stores assumptions, Plan records an accepted intervention, and Job tracks its execution. threshold_action is a readable rule-based comparator.

The simulator uses synthetic parameters and generated random tapes. They make comparisons repeatable under a seed, but they are not fitted engine measurements. Exposure is a sampled aircraft-level multiplier in this environment: it changes modeled wear and sensor temperature, and should not be read as a measured real-world quantity. _update_filter updates the policy-facing belief from noisy sensor readings; it does not receive the simulator's true event labels or hidden health. step decides which aircraft are grounded, schedules accepted jobs, simulates flights and hazards for the others, completes repairs, updates metrics, and returns the next observation.

8.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
# fleet_maintenance/fleet_env.py
# ======================================================================


from dataclasses import dataclass
import math
import numpy as np
import gymnasium as gym
from gymnasium import spaces
COMPONENTS = ('fan', 'LPC', 'HPC', 'combustor', 'HPT', 'LPT')
WEAR = np.array([0.002, 0.0025, 0.004, 0.002, 0.003, 0.0025])
LABOR_DAYS = np.array([2, 3, 4, 2, 3, 3])
PART_COST = np.array([8.0, 10.0, 16.0, 8.0, 12.0, 10.0])
EFFICIENCY_BASE = 0.72
EFFICIENCY_GAIN = 0.24
TEMPERATURE_BASE_C = np.array([40.0, 120.0, 350.0, 900.0, 650.0, 450.0])
TEMPERATURE_HEALTH_GAIN_C = 120.0
TEMPERATURE_EXPOSURE_GAIN_C = 30.0
TEMPERATURE_MIN_C = -50.0
TEMPERATURE_MAX_C = 1400.0
OBSERVATION_VERSION = 2

```

Line-by-line explanation.

242 This source comment labels or clarifies the section; Python does not execute it.

243 This comment draws a visual boundary between source sections. It has no runtime effect.

246 Generate record-class methods from annotated fields.

247 Use operations such as ceiling when calculating maintenance duration.

Use NumPy for arrays, vectorized simulation, random sampling, and numeric calculations.

Use Gymnasium environment/wrapper base classes under the conventional alias.

Declare and validate legal observation and action shapes.

Define COMPONENTS as a shared modeling constant: the six component names in bundle-bit order.

Define WEAR as a shared modeling constant: baseline daily wear for each component.

Define LABOR_DAYS as a shared modeling constant: labor duration for each component replacement.

Define PART_COST as a shared modeling constant: replacement-part cost by component type.

Define EFFICIENCY_BASE as a shared modeling constant: efficiency baseline at zero health.

Define EFFICIENCY_GAIN as a shared modeling constant: efficiency change from health.

Define TEMPERATURE_BASE_C as a shared modeling constant: baseline temperature by component.

Define shared module constant TEMPERATURE_HEALTH_GAIN_C; other examples reuse this value.

Define shared module constant TEMPERATURE_EXPOSURE_GAIN_C; other examples reuse this value.

Define shared module constant TEMPERATURE_MIN_C; other examples reuse this value.

Define shared module constant TEMPERATURE_MAX_C; other examples reuse this value.

Define OBSERVATION_VERSION as a shared modeling constant: saved observation-layout version tag.

8.2. Class `Config`

This class an immutable record of fleet, horizon, resources, disturbances, and estimation settings.

```python
@dataclass(frozen=True)
class Config:
    aircraft: int = 7
    horizon: int = 90
    crews: int = 3
    workshops: int = 2
    particles: int = 64
    setup_days: int = 7
    replenish_days: int = 14
    bird_probability: float = 0.002
    hail_probability: float = 0.003
    weather_probability: float = 0.03
    observation_noise: float = 0.025
    wear_scale: float = 1.0
    mandatory_health: float = 0.08
    initial_health: float = 0.88

```

Line-by-line explanation.

264 Apply dataclass's constructor/repr/equality behavior and freeze field reassignment; these instances are immutable records.

265 Define class Config: an immutable record of fleet, horizon, resources, disturbances, and estimation settings.

266 Declare Config field aircraft as int with default 7; it controls fleet size.

267 Declare Config field horizon as int with default 90; it controls episode duration in days.

268 Declare Config field crews as int with default 3; it controls available maintenance crews.

269 Declare Config field workshops as int with default 2; it controls parallel maintenance shops.

270 Declare Config field particles as int with default 64; it controls health-belief samples per component.

271 Declare Config field setup_days as int with default 7; it controls fixed setup delay before component work.

272 Declare Config field replenish_days as int with default 14; it controls inventory replenishment interval.

273 Declare Config field bird_probability as float with default 0.002; it controls bird-event probability.

274 Declare Config field hail_probability as float with default 0.003; it controls hail-event probability.

275 Declare Config field weather_probability as float with default 0.03; it controls weather-event probability.

276 Declare Config field observation_noise as float with default 0.025; it controls sensor noise scale.

277 Declare Config field wear_scale as float with default 1.0; it controls wear multiplier.

278 Declare Config field mandatory_health as float with default 0.08; it controls safety threshold for estimated lower health.

279 Declare Config field initial_health as float with default 0.88; it controls mean component health at reset.

8.2.1 `Config.__post_init__`

The dataclass-generated initializer calls this immediately after assigning fields, so invalid configuration is rejected before an environment is built.

```
281     def __post_init__(self):
282         for k in ('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days',
        ↪  'replenish_days'):
283             if not isinstance(getattr(self, k), int) or getattr(self, k) < 1:
284                 raise ValueError(f'{k} must be a positive integer')
285         for k in ('bird_probability', 'hail_probability', 'weather_probability'):
286             if not 0 <= getattr(self, k) <= 1:
287                 raise ValueError(f'{k} must be in [0,1]')
288         if not 0 < self.observation_noise or not 0 < self.wear_scale:
289             raise ValueError('noise and wear_scale must be positive')
```

```
290         if not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1:
291             raise ValueError('invalid health bounds')
```

Line-by-line explanation.

281 Define ___post_init___ with the shown parameters/defaults; its purpose is to validate dataclass fields after the generated initializer has assigned them.

282 Loop through ('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days', 'replenish_days'), handling each k once.

283 Validate the input invariant not isinstance(getattr(self, k), int) or getattr(self, k) < 1; the guard rejects invalid values before the state-changing code runs.

284 Raise ValueError(f'{k} must be a positive integer'); invalid input stops here instead of creating inconsistent state.

285 Loop through ('bird_probability', 'hail_probability', 'weather_probability'), handling each k once.

286 Validate the input invariant not 0 <= getattr(self, k) <= 1; the guard rejects invalid values before the state-changing code runs.

287 Raise ValueError(f'{k} must be in [0,1]'); invalid input stops here instead of creating inconsistent state.

288 Validate the input invariant not 0 < self.observation_noise or not 0 < self.wear_scale; the guard rejects invalid values before the state-changing code runs.

289 Raise ValueError('noise and wear_scale must be positive'); invalid input stops here instead of creating inconsistent state.

290 Validate the input invariant not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1; the guard rejects invalid values before the state-changing code runs.

291 Raise ValueError('invalid health bounds'); invalid input stops here instead of creating inconsistent state.

8.3. Class `Plan`

This class a record of an accepted plan and its engine/component selections and resources.

```
293 @dataclass(frozen=True)
294 class Plan:
295     aircraft: int
296     engine_mask: int
297     bundles: tuple[int, int]
298     crews: int
299     workshop: int
300     order_start: int
301     priority: int
```

Line-by-line explanation.

293 Apply dataclass's constructor/repr/equality behavior and freeze field reassignment; these instances are immutable records.

294 Define class Plan: a record of an accepted plan and its engine/component selections and resources.

295 Declare record field aircraft as int; dataclass construction and attribute access carry fleet size.

296 Declare record field engine_mask as int; dataclass construction and attribute access carry engine-selection bitmask.

297 Declare record field bundles as tuple[int, int]; dataclass construction and attribute access carry the two per-engine component masks.

298 Declare record field crews as int; dataclass construction and attribute access carry available maintenance crews.

299 Declare record field workshop as int; dataclass construction and attribute access carry this part of the plan/job.

300 Declare record field order_start as int; dataclass construction and attribute access carry this part of the plan/job.

301 Declare record field priority as int; dataclass construction and attribute access carry this part of the plan/job.

8.4. Class `Job`

This class the live execution record of a plan, its tasks, and remaining duration.

```
303 @dataclass
304 class Job:
305     plan: Plan
306     tasks: tuple[tuple[int, int], ...]
307     remaining: int
308     total_duration: int
```

Line-by-line explanation.

303 Have dataclass generate the usual record methods while allowing instances' fields to change.

304 Define class Job: the live execution record of a plan, its tasks, and remaining duration.

305 Declare record field plan as Plan; dataclass construction and attribute access carry this part of the plan/job.

306 Declare record field tasks as tuple[tuple[int, int], ...]; dataclass construction and attribute access carry selected (engine, component) repair pairs.

307 Declare record field remaining as int; dataclass construction and attribute access carry days left in this job.

308 Declare record field total_duration as int; dataclass construction and attribute access carry the job's full planned duration.

8.5. Class `FleetMaintenanceEnv`

This class the full simulator for hidden health, noisy observations, resources, jobs, rewards, and events.

```
310 class FleetMaintenanceEnv(gym.Env):
311     """Action row: ground, engines, bundle0, bundle1, crews-1, shop, order, priority.
312
```

```
313     Engine mask bits 0/1 select engines. Each bundle has six component bits.
314     Resources are assigned to complete plans, in selected priority order.
315     Unfunded/conflicting proposals are rejected and logged; no partial repairs.
316     A busy aircraft's row is ignored: the committed plan cannot be overwritten.
317     Mandatory grounding uses observed lower health quantiles, not hidden health.
318     """
319     metadata = {'render_modes': ['ansi']}
320
```

Line-by-line explanation.

310 Define class FleetMaintenanceEnv: the full simulator for hidden health, noisy observations, resources, jobs, rewards, and events.

311 This docstring states FleetMaintenanceEnv's purpose/contract: Action row: ground, engines, bundle0, bundle1, crews-1, shop, order, priority. It is documentation text, not an executable operation.

313 This docstring line states: Engine mask bits 0/1 select engines. Each bundle has six component bits.

314 This docstring line states: Resources are assigned to complete plans, in selected priority order.

315 This docstring line states: Unfunded/conflicting proposals are rejected and logged; no partial repairs.

316 This docstring line states: A busy aircraft's row is ignored: the committed plan cannot be overwritten.

317 This docstring line states: Mandatory grounding uses observed lower health quantiles, not hidden health.

318 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

319 Store the result in metadata so later lines can use it.

8.5.1 `FleetMaintenanceEnv.__init__`

This sets the configuration, observation and action spaces, feature widths, and initial episode flag. The policy constructors read these spaces and dimensions to build compatible networks.

```
321     def __init__(self, config: Config | None=None, render_mode=None):
322         super().__init__()
323         self.config = config or Config()
324         self.render_mode = render_mode
325         n = self.config.aircraft
326         self.row_dims = np.array([2, 4, 64, 64, min(2, self.config.crews), self.config.workshops, 12,
            ↪  n])
327         self.action_space = spaces.MultiDiscrete(np.tile(self.row_dims, (n, 1)))
328         self.local_size = 70
329         self.public_size = 10
330         self.observation_space = spaces.Box(0.0, 1.0, shape=(n * self.local_size + self.public_size,),
            ↪  dtype=np.float32)
331         self._done = True
```

Line-by-line explanation.

321 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

322 Initialize the parent class before setting subclass-specific state.

323 Store this value as self.config on the object so later method calls can read it.

324 Store this value as self.render_mode on the object so later method calls can read it.

325 Store the result in n so later lines can use it.

326 Store this value as self.row_dims on the object so later method calls can read it.

327 Store this value as self.action_space on the object so later method calls can read it.

328 Store this value as self.local_size on the object so later method calls can read it.

329 Store this value as self.public_size on the object so later method calls can read it.

330 Store this value as self.observation_space on the object so later method calls can read it.

331 Store this value as self._done on the object so later method calls can read it.

8.5.2 `FleetMaintenanceEnv.reset`

train and evaluate call this before each episode. It initializes separate random streams, hidden health, event tapes, inventory, jobs, metrics, noisy measurements, and the initial belief-state observation.

```
333     def reset(self, *, seed=None, options=None):
334         super().reset(seed=seed)
335         cfg = self.config
336         seeds = np.random.SeedSequence(seed).spawn(3)
337         self.physical_rng = np.random.default_rng(seeds[0])
338         self.filter_rng = np.random.default_rng(seeds[1])
339         tape_rng = np.random.default_rng(seeds[2])
340         self.day = 0
341         self.health = np.clip(cfg.initial_health + self.physical_rng.normal(0, 0.025, (cfg.aircraft, 2,
            ↪  6)), 0.1, 1)
342         if options and 'health' in options:
343             health = np.asarray(options['health'], dtype=float)
344             if health.shape != self.health.shape or np.any((health < 0) | (health > 1)):
345                 raise ValueError('initial health must have shape (aircraft,2,6), in [0,1]')
346             self.health = health.copy()
347         self.exposure = tape_rng.uniform(0.75, 1.35, cfg.aircraft)
348         self.measurements = self._sensor_model(self.health, self.exposure[:, None, None])
349         self.measurements += self.physical_rng.normal(0, 1, self.measurements.shape) *
            ↪  self.sensor_noise
350         self._clip_measurements(self.measurements)
351         self.last_flight_day = np.zeros(cfg.aircraft, dtype=int)
352         prior = self._invert_sensor_metrics(self.measurements, self.exposure[:, None, None])
353         self.particles = np.clip(prior[..., None] + self.filter_rng.normal(0, 0.04,
            ↪  (*self.health.shape, cfg.particles)), 0, 1)
354         self.jobs: dict[int, Job] = {}
355         self.stock = np.full(6, cfg.aircraft, dtype=int)
356         self.event_tape = tape_rng.random((cfg.horizon, cfg.aircraft, 2, 2))
357         self.severity_tape = tape_rng.uniform(0.1, 0.35, (cfg.horizon, cfg.aircraft, 2, 2))
358         self.weather_tape = tape_rng.random(cfg.horizon)
359         self.wear_tape = tape_rng.normal(1.0, 0.12, (cfg.horizon, cfg.aircraft, 2, 6))
360         self.noise_tape = tape_rng.normal(0, cfg.observation_noise, (cfg.horizon, cfg.aircraft, 2, 6,
            ↪  2))
361         self.metrics = {k: 0.0 for k in ('return', 'cost', 'flight_failures', 'flights',
            ↪  'downtime_aircraft_days', 'groundings', 'replacements', 'rejections', 'bird_events',
            ↪  'hail_events', 'weather_exposures', 'unsafe_attempts')}
362         self._done = False
363         return (self._observation(), {'day': self.day})
```

Line-by-line explanation.

333 Define reset with the shown parameters/defaults; its purpose is to start an episode and initialize its random streams and state.

334 Store the result in super().reset(seed so later lines can use it.

335 Store the result in cfg so later lines can use it.

336 Store the result in seeds so later lines can use it.

337 Store this value as self.physical_rng on the object so later method calls can read it.

338 Store this value as self.filter_rng on the object so later method calls can read it.

339 Store the result in tape_rng so later lines can use it.

340 Store this value as self.day on the object so later method calls can read it.

341 Update self.health. it holds health values held by this toy state or simulator for later transitions.

342 When options and 'health' in options is true, enter this block; otherwise continue to the next case.

343 Convert the expression to a NumPy array and store it as health (health values held by this toy state or simulator).

344 When health.shape != self.health.shape or np.any((health < 0) | (health > 1)) is true, enter this block; otherwise continue to the next case.

345 Raise ValueError('initial health must have shape (aircraft,2,6), in [0,1]'); invalid input stops here instead of creating inconsistent state.

346 Update self.health. it holds health values held by this toy state or simulator for later transitions.

347 Compute the aircraft-level multiplier that changes modeled wear and heat and store it in self.exposure; later checks/updates use this named intermediate.

348 Update self.measurements. it holds noisy sensor readings, not hidden true health for later transitions.

349 Add self.physical_rng.normal(0, 1, self.measurements.shape) * self.sensor_noise to the running total in self.measurements.

350 Call self._clip_measurements to keep sensor values inside the supported ranges.

351 Store this value as self.last_flight_day on the object so later method calls can read it.

352 Store the result in prior so later lines can use it.

353 Update self.particles. it holds sampled health beliefs maintained by the filter for later transitions.

354 Separate this result into self.jobs: dict[int, Job] for the steps that follow.

355 Create self.stock as a fixed-shape array filled with the given starting value; it represents the spare-part inventory by component type.

356 Store this value as self.event_tape on the object so later method calls can read it.

357 Store this value as self.severity_tape on the object so later method calls can read it.

358 Store this value as self.weather_tape on the object so later method calls can read it.

359 Store this value as self.wear_tape on the object so later method calls can read it.

360 Store this value as self.noise_tape on the object so later method calls can read it.

361 Update self.metrics. it holds the accumulated simulator performance counters for later transitions.

362 Store this value as self._done on the object so later method calls can read it.

363 Return the current observation, reward, terminal flags, and info in the API's documented order.

8.5.3 `FleetMaintenanceEnv.split_observation`

Policy and baseline helpers call this to separate per-aircraft rows from public fleet values, so each decision can use local condition and shared resource information in the proper shape.

```
365     def split_observation(self, observation):
366         a = np.asarray(observation)
367         return (a[:-self.public_size].reshape(self.config.aircraft, self.local_size),
        ↪  a[-self.public_size:])
```

Line-by-line explanation.

365 Define split_observation with the shown parameters/defaults; its purpose is to separate each aircraft's local features from fleet-wide public values.

366 Store the result in a so later lines can use it.

367 Return these values together; callers unpack them in the order shown.

8.5.4 `FleetMaintenanceEnv.idle_action`

threshold_action and execute_option use this to start from a valid no-new-work proposal before adding planned maintenance rows.

```
369     def idle_action(self):
370         return np.zeros((self.config.aircraft, 8), dtype=np.int64)
```

Line-by-line explanation.

369 Define idle_action with the shown parameters/defaults; its purpose is to create the all-zero action matrix used when no plan is proposed.

370 Return the value produced by this expression to the caller.

8.5.5 `FleetMaintenanceEnv.resource_state`

The observation builder, job-start logic, and text renderer call this to report free crews and workshops consistently.

```
372     def resource_state(self):
373         used = sum((j.plan.crews for j in self.jobs.values()))
374         busy_shops = {j.plan.workshop for j in self.jobs.values()}
375         return (self.config.crews - used, [s for s in range(self.config.workshops) if s not in
        ↪  busy_shops])
```

Line-by-line explanation.

372 Define resource_state with the shown parameters/defaults; its purpose is to count uncommitted crews and list unoccupied workshops.

373 Store the result in used so later lines can use it.

374 Store the result in busy_shops so later lines can use it.

375 Return these values together; callers unpack them in the order shown.

8.5.6 `FleetMaintenanceEnv.sensor_noise`

Initialization uses this property to scale noise added to synthetic efficiency and temperature readings.

```
377     @property
378     def sensor_noise(self):
379         """Independent sensor noise scales in efficiency fraction and Celsius."""
380         return np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) * self.config.observation_noise
```

Line-by-line explanation.

377 Make the following method readable as an attribute, so callers do not add parentheses.

378 Define sensor_noise with the shown parameters/defaults; its purpose is to return the noise scale used for efficiency and temperature channels.

379 This docstring states sensor_noise's purpose/contract: Independent sensor noise scales in efficiency fraction and Celsius. It is documentation text, not an executable operation.

380 Return the value produced by this expression to the caller.

8.5.7 `FleetMaintenanceEnv._sensor_model`

reset() and step() use this to map hidden component health and exposure into synthetic efficiency and temperature measurements.

```
382     @staticmethod
383     def _sensor_model(health, exposure):
384         """Toy performance mapping; health decreases efficiency and raises heat."""
385         efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * health
386         temperature = TEMPERATURE_BASE_C + TEMPERATURE_HEALTH_GAIN_C * (1 - health) +
            ↪  TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)
387         return np.stack([efficiency, temperature], axis=-1)
```

Line-by-line explanation.

382 Apply decorator staticmethod to the definition below; Python passes that class/function through the decorator.

383 Define _sensor_model with the shown parameters/defaults; its purpose is to map hidden health/exposure into synthetic efficiency and temperature readings.

384 This docstring states _sensor_model's purpose/contract: Toy performance mapping; health decreases efficiency and raises heat. It is documentation text, not an executable operation.

385 Store the result in efficiency so later lines can use it.

386 Store the result in temperature so later lines can use it.

387 Return the value produced by this expression to the caller.

8.5.8 `FleetMaintenanceEnv._clip_measurements`

reset() and step() call this after sensor generation so measurements stay within the ranges represented by the observation model.

```
389     @staticmethod
390     def _clip_measurements(measurements):
391         np.clip(measurements[..., 0], 0, 1, out=measurements[..., 0])
392         np.clip(measurements[..., 1], TEMPERATURE_MIN_C, TEMPERATURE_MAX_C, out=measurements[..., 1])
```

Line-by-line explanation.

389 Apply decorator staticmethod to the definition below; Python passes that class/function through the decorator.

390 Define _clip_measurements with the shown parameters/defaults; its purpose is to keep sensor values within their modeled bounds.

391 Separate this result into np.clip(measurements[..., 0], 0, 1, out for the steps that follow.

392 Separate this result into np.clip(measurements[..., 1], TEMPERATURE_MIN_C, TEMPERATURE_MAX_C, out for the steps that follow.

8.5.9 `FleetMaintenanceEnv._invert_sensor_metrics`

reset() uses this to turn the initial noisy sensor values into the prior particles for the belief filter.

```
394     @staticmethod
395     def _invert_sensor_metrics(measurements, exposure):
396         """Initialization only: fuse two noisy inverse mappings with equal weight."""
397         from_efficiency = (measurements[..., 0] - EFFICIENCY_BASE) / EFFICIENCY_GAIN
398         from_temperature = 1 - (measurements[..., 1] - TEMPERATURE_BASE_C - TEMPERATURE_EXPOSURE_GAIN_C
            ↪  * (exposure - 1)) / TEMPERATURE_HEALTH_GAIN_C
399         return np.clip(0.5 * (from_efficiency + from_temperature), 0, 1)
```

Line-by-line explanation.

394 Apply decorator staticmethod to the definition below; Python passes that class/function through the decorator.

395 Define _invert_sensor_metrics with the shown parameters/defaults; its purpose is to estimate health from the two sensor equations at initialization.

396 This docstring states _invert_sensor_metrics's purpose/contract: Initialization only: fuse two noisy inverse mappings with equal weight. It is documentation text, not an executable operation.

397 Store the result in from_efficiency so later lines can use it.

398 Store the result in from_temperature so later lines can use it.

399 Return the value produced by this expression to the caller.

8.5.10 `FleetMaintenanceEnv.observation_metrics`

This utility decodes normalized observation columns into named measurements for analysis; it does not change the simulator state.

```
401     def observation_metrics(self, observation):
402         """Decode observed sensor metrics without exposing hidden health."""
403         rows, _ = self.split_observation(observation)
404         return {'efficiency': rows[:, 45:57].reshape(self.config.aircraft, 2, 6).copy(),
            ↪  'temperature_c': rows[:, 57:69].reshape(self.config.aircraft, 2, 6) * (TEMPERATURE_MAX_C -
            ↪  TEMPERATURE_MIN_C) + TEMPERATURE_MIN_C, 'reading_age_days': np.rint(rows[:, 69] *
            ↪  self.config.horizon).astype(int)}
```

Line-by-line explanation.

401 Define observation_metrics with the shown parameters/defaults; its purpose is to decode sensor columns without exposing simulator hidden health.

402 This docstring states observation_metrics's purpose/contract: Decode observed sensor metrics without exposing hidden health. It is documentation text, not an executable operation.

403 Unpack the return values from self.split_observation(observation) into rows; underscore entries are intentionally ignored.

404 Return these values together; callers unpack them in the order shown.

8.5.11 `FleetMaintenanceEnv._observation`

reset and step call this to convert hidden simulator state and the particle filter into the public vector. The policy receives means, uncertainty summaries, lower health quantiles, job status, resources, sensor features, and ages, not the true health array.

```
406     def _observation(self):
407         mean = self.particles.mean(-1)
408         std = self.particles.std(-1)
409         low = np.quantile(self.particles, 0.1, axis=-1)
410         rows = []
411         for a in range(self.config.aircraft):
412             j = self.jobs.get(a)
413             committed = [j.plan.engine_mask / 3, j.plan.bundles[0] / 63, j.plan.bundles[1] / 63,
                ↪  j.plan.crews / self.config.crews, j.plan.workshop / max(1, self.config.workshops - 1),
                ↪  j.plan.order_start / 11] if j else [0.0] * 6
414             rows.append(np.concatenate([mean[a].ravel(), std[a].ravel(), low[a].ravel(),
                ↪  [self.exposure[a] / 1.5, float(j is not None), min(1, j.remaining / 60) if j else 0],
                ↪  committed, self.measurements[a, :, :, 0].ravel(), ((self.measurements[a, :, :, 1] -
                ↪  TEMPERATURE_MIN_C) / (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C)).ravel(), [(self.day -
                ↪  self.last_flight_day[a]) / self.config.horizon]]))
415         free, shops = self.resource_state()
416         public = [self.day / self.config.horizon, free / self.config.crews, len(shops) /
            ↪  self.config.workshops, (self.config.replenish_days - self.day % self.config.replenish_days)
            ↪  / self.config.replenish_days, *np.clip(self.stock / self.config.aircraft, 0, 1)]
417         return np.asarray(np.concatenate([np.asarray(rows).ravel(), public]), dtype=np.float32)
```

Line-by-line explanation.

406 Define _observation with the shown parameters/defaults; its purpose is to pack internal state into the public observation layout.

407 Store the result in mean so later lines can use it.

408 Store the result in std so later lines can use it.

409 Compute the conservative 10th-percentile health estimate and store it in low; later checks/updates use this named intermediate.

410 Store the result in rows so later lines can use it.

411 Loop through range(self.config.aircraft), handling each a once.

412 Store the result in j so later lines can use it.

413 Store the result in committed so later lines can use it.

414 Add this item to rows so it stays available for later use.

415 Separate this result into free, shops for the steps that follow.

416 Compute fleet-wide observation values shared by aircraft and store it in public; later checks/updates use this named intermediate.

417 Return the assembled observation as a NumPy float32 vector.

8.5.12 `FleetMaintenanceEnv.validate_row`

step calls this before accepting a new plan. It catches structural inconsistencies between the grounding flag, engine mask, and component bundles.

```
419     def validate_row(self, aircraft, row):
420         ground, engines, b0, b1, crew, shop, order, priority = map(int, row)
421         if not ground:
422             return None if engines == b0 == b1 == 0 else 'work_without_grounding'
423         actual = int(b0 != 0) + 2 * int(b1 != 0)
424         if not actual or engines != actual:
425             return 'inconsistent_engine_bundle'
426         return None
```

Line-by-line explanation.

419 Define validate_row with the shown parameters/defaults; its purpose is to check one aircraft row's grounding/engine/bundle consistency.

420 Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

421 When not ground is true, enter this block; otherwise continue to the next case.

422 Return the value produced by this expression to the caller.

423 Derive the engine mask from whether each bundle is nonempty. int(b0 != 0) is engine 0's low bit (1); 2 * int(b1 != 0) places engine 1 in bit 1 (2). Examples: [5,0]→01_2→1, [0,8]→10_2→2, [5,8]→11_2→3. Both zero gives 0 and cannot describe grounded work. The factor 2 is a bit-position weight, not extra priority; a busy aircraft's committed job is separate and is not canceled by this proposal.

424 Reject a grounded work row if both bundles are empty, or if the engine mask disagrees with which bundles are nonzero.

425 Return 'inconsistent_engine_bundle' to the caller; this ends the function and supplies its result.

426 Return None to signal that validation found no error; callers treat this as the successful case.

8.5.13 `FleetMaintenanceEnv._start_job`

step calls this for a structurally valid proposed plan. It checks crew, shop, and spare availability before reserving resources and creating a committed Job.

```
428     def _start_job(self, aircraft, row):
429         _, engines, b0, b1, crew, shop, order, priority = map(int, row)
430         plan = Plan(aircraft, engines, (b0, b1), crew + 1, shop, order, priority)
431         tasks = [(e, c) for e, b in enumerate((b0, b1)) for c in range(6) if b & 1 << c]
432         tasks.sort(key=lambda x: (x[0] * 6 + x[1] - order) % 12)
433         needed = np.bincount([c for _, c in tasks], minlength=6)
434         free, shops = self.resource_state()
435         if plan.crews > free or shop not in shops:
436             return ('resource_conflict', 0.0)
437         if np.any(needed > self.stock):
438             return ('spare_shortage', 0.0)
439         self.stock -= needed
440         duration = self.config.setup_days + sum((math.ceil(LABOR_DAYS[c] / plan.crews) for _, c in
        ↪  tasks))
441         self.jobs[aircraft] = Job(plan, tuple(tasks), duration, duration)
442         cost = 12.0 + engines.bit_count() * 5 + sum((PART_COST[c] + 3 * self.health[aircraft, e, c] for
        ↪  e, c in tasks))
443         self.metrics['groundings'] += 1
444         return (None, cost)
```

Line-by-line explanation.

428 Define _start_job with the shown parameters/defaults; its purpose is to check resources/spares and commit an accepted maintenance plan.

429 Unpack the return values from map(int, row) into engines, b0, b1, crew, shop, order, priority; underscore entries are intentionally ignored.

430 Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

431 Decode bundle bits into (engine, component) tasks. enumerate maps b0/b1 to engine e; b & (1 « c) tests component c. Each set bit adds one repair task; a zero bit adds none.

432 Sort component tasks cyclically from the requested start position. The engine/component pair maps to one of 12 positions, and modulo 12 wraps the ordering around.

433 Count requested repairs by component type, then compare those needs with inventory before committing anything; this prevents partial plans when stock is short.

434 Separate this result into free, shops for the steps that follow.

435 When plan.crews > free or shop not in shops is true, enter this block; otherwise continue to the next case.

436 Return these values together; callers unpack them in the order shown.

437 When np.any(needed > self.stock) is true, enter this block; otherwise continue to the next case.

438 Return these values together; callers unpack them in the order shown.

439 Subtract needed from self.stock to account for work or wear during this transition.

440 Compute the primitive days consumed by a job or option and store it in duration; later checks/updates use this named intermediate.

441 Update self.jobs[aircraft]. it holds accepted maintenance plans still in progress for later transitions.

442 Store the result in cost so later lines can use it.

443 Add 1 to the running total in self.metrics['groundings'].

444 Return these values together; callers unpack them in the order shown.

8.5.14 `FleetMaintenanceEnv._update_filter`

step calls this for aircraft that fly, using measurements to update particles. It is the simulator's observation-estimation path; it deliberately does not pass true event type or severity to the filter.

```
446     def _update_filter(self, aircraft, measurements):
447         """Bootstrap particle filter with noisy observations and shock-mixture prior.
448 
449         The estimator does not receive true event type, severity or hidden health.
450         It is a deliberately simple independent-component filter, not Emanuele's.
451         """
452         cfg = self.config
453         p = self.particles[aircraft]
454         wear = WEAR[None, :, None] * self.exposure[aircraft] * cfg.wear_scale
455         p -= wear * self.filter_rng.normal(1, 0.12, p.shape)
456         bird = self.filter_rng.random((2, 1, cfg.particles)) < cfg.bird_probability
457         hail = self.filter_rng.random((2, 1, cfg.particles)) < cfg.hail_probability
458         bseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
459         hseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
460         p -= bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None]
461         p -= hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None]
462         p -= (self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2
463         np.clip(p, 0, 1, out=p)
464         predicted_efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * p
465         predicted_temperature = TEMPERATURE_BASE_C[None, :, None] + TEMPERATURE_HEALTH_GAIN_C * (1 - p)
        ↪  + TEMPERATURE_EXPOSURE_GAIN_C * (self.exposure[aircraft] - 1)
466         efficiency_error = (predicted_efficiency - measurements[:, :, 0, None]) / self.sensor_noise[0]
467         temperature_error = (predicted_temperature - measurements[:, :, 1, None]) /
        ↪  self.sensor_noise[1]
468         logits = -0.5 * (efficiency_error ** 2 + temperature_error ** 2)
469         weights = np.exp(logits - logits.max(-1, keepdims=True))
470         weights /= weights.sum(-1, keepdims=True)
471         for e in range(2):
472             for c in range(6):
473                 idx = self.filter_rng.choice(cfg.particles, cfg.particles, p=weights[e, c])
474                 p[e, c] = np.clip(p[e, c, idx] + self.filter_rng.normal(0, 0.006, cfg.particles), 0, 1)
```

Line-by-line explanation.

446 Define _update_filter with the shown parameters/defaults; its purpose is to advance health-belief particles and reweight them from sensor evidence.

447 This docstring states _update_filter's purpose/contract: Bootstrap particle filter with noisy observations and shock-mixture prior. It is documentation text, not an executable operation.

449 This docstring line states: The estimator does not receive true event type, severity or hidden health.

450 This docstring line states: It is a deliberately simple independent-component filter, not Emanuele's.

451 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

452 Store the result in cfg so later lines can use it.

453 Store the result in p so later lines can use it.

454 Store the result in wear so later lines can use it.

455 Subtract wear * self.filter_rng.normal(1, 0.12, p.shape) from p to account for work or wear during this transition.

456 Store the result in bird so later lines can use it.

457 Store the result in hail so later lines can use it.

458 Store the result in bseverity so later lines can use it.

459 Store the result in hseverity so later lines can use it.

460 Subtract bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None] from p to account for work or wear during this transition.

461 Subtract hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None] from p to account for work or wear during this transition.

462 Subtract (self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2 from p to account for work or wear during this transition.

463 Separate this result into np.clip(p, 0, 1, out for the steps that follow.

464 Store the result in predicted_efficiency so later lines can use it.

465 Store the result in predicted_temperature so later lines can use it.

466 Store the result in efficiency_error so later lines can use it.

467 Store the result in temperature_error so later lines can use it.

468 Compute unnormalized scores later converted to choice probabilities and store it in logits; later checks/updates use this named intermediate.

469 Store the result in weights so later lines can use it.

470 Store the result in weights so later lines can use it.

471 Loop through range(2), handling each e once.

472 Loop through range(6), handling each c once.

473 Store the result in idx so later lines can use it.

474 Separate this result into p[e, c] for the steps that follow.

8.5.15 `FleetMaintenanceEnv.step`

This is the simulator's central transition. train and evaluate call it directly for daily policies, and execute_option calls it repeatedly while a manager-level decision remains active.

```
476     def step(self, action):
477         if self._done:
478             raise RuntimeError('reset before step, including after episode completion')
479         a = np.asarray(action)
480         if not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a):
481             raise ValueError('action outside MultiDiscrete space')
482         cfg = self.config
483         low = np.quantile(self.particles, 0.1, axis=-1)
484         mandatory = low.min(axis=(1, 2)) <= cfg.mandatory_health
485         costs = 0.0
486         events = []
```

```
        rejected = []
        completed = []
        grounded = set(self.jobs)
        for aircraft in sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i)):
            if aircraft in self.jobs:
                continue
            reason = self.validate_row(aircraft, a[aircraft])
            if reason is None and int(a[aircraft, 0]):
                reason, cost = self._start_job(aircraft, a[aircraft])
                costs += cost
            if reason:
                rejected.append((aircraft, reason))
                costs += 2.0
            if mandatory[aircraft] and aircraft not in self.jobs:
                grounded.add(aircraft)
                self.metrics['unsafe_attempts'] += float(not int(a[aircraft, 0]))
            if aircraft in self.jobs:
                grounded.add(aircraft)
        self.metrics['rejections'] += len(rejected)
        flights = 0
        failures = 0
        weather = self.weather_tape[self.day] < cfg.weather_probability
        for aircraft in range(cfg.aircraft):
            if aircraft in grounded:
                continue
            flights += 1
            self.health[aircraft] -= WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0,
            ↪  self.wear_tape[self.day, aircraft]) * (3 if weather else 1)
            if weather:
                self.metrics['weather_exposures'] += 1
            for e in range(2):
                for kind, p, impact in [('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]),
                ↪  ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])]:
                    k = int(kind == 'hail')
                    if self.event_tape[self.day, aircraft, e, k] < p:
                        severity = self.severity_tape[self.day, aircraft, e, k]
                        self.health[aircraft, e] -= severity * np.array(impact)
                        events.append({'aircraft': aircraft, 'engine': e, 'type': kind})
                        self.metrics[kind + '_events'] += 1
            np.clip(self.health[aircraft], 0, 1, out=self.health[aircraft])
            failed = bool(np.any(self.health[aircraft] <= 0))
            failures += failed
            self.measurements[aircraft] = self._sensor_model(self.health[aircraft],
            ↪  self.exposure[aircraft])
            self.measurements[aircraft] += self.noise_tape[self.day, aircraft] *
            ↪  np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C])
            self._clip_measurements(self.measurements[aircraft])
            self.last_flight_day[aircraft] = self.day + 1
            self._update_filter(aircraft, self.measurements[aircraft])
        for aircraft, j in list(self.jobs.items()):
            elapsed = j.total_duration - j.remaining + 1
            task_day = cfg.setup_days
            for e, c in j.tasks:
                task_day += math.ceil(LABOR_DAYS[c] / j.plan.crews)
                if elapsed == task_day:
                    self.health[aircraft, e, c] = 1.0
                    self.particles[aircraft, e, c] = 1.0
                    self.metrics['replacements'] += 1
            j.remaining -= 1
            if j.remaining == 0:
                completed.append(aircraft)
                del self.jobs[aircraft]
        downtime = len(grounded)
        costs += downtime * 2 + failures * 80
        reward = (flights * 10 - costs) / (cfg.aircraft * 10)
        self.metrics['return'] += reward
        self.metrics['cost'] += costs
        self.metrics['flights'] += flights
        self.metrics['flight_failures'] += failures
        self.metrics['downtime_aircraft_days'] += downtime
        self.day += 1
        if self.day % cfg.replenish_days == 0:
            self.stock = np.minimum(cfg.aircraft, self.stock + max(1, cfg.aircraft // 2))
        self._done = self.day >= cfg.horizon
```

```
557         info = {'day': self.day, 'events': events, 'rejected': rejected, 'completed': completed,
            ↪  'reward_terms': {'flight_revenue': flights * 10, 'cost': costs}, 'metrics':
            ↪  dict(self.metrics)}
558         return (self._observation(), float(reward), self._done, False, info)
```

Line-by-line explanation.

476 Define step with the shown parameters/defaults; its purpose is to validate one action, advance the simulation, and return the transition.

477 Check the episode/option stop condition self._done; when true, stop collecting transitions for this segment.

478 Raise RuntimeError('reset before step, including after episode completion'); invalid input stops here instead of creating inconsistent state.

479 Store the result in a so later lines can use it.

480 When not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a) is true, enter this block; otherwise continue to the next case.

481 Raise ValueError('action outside MultiDiscrete space'); invalid input stops here instead of creating inconsistent state.

482 Store the result in cfg so later lines can use it.

483 Compute the conservative 10th-percentile health estimate and store it in low; later checks/updates use this named intermediate.

484 Compute whether a lower-health safety alarm is active and store it in mandatory; later checks/updates use this named intermediate.

485 Compute repair, downtime, rejection, and failure costs and store it in costs; later checks/updates use this named intermediate.

486 Compute event records generated during the transition and store it in events; later checks/updates use this named intermediate.

487 Compute proposed plans that were not accepted and why and store it in rejected; later checks/updates use this named intermediate.

488 Compute aircraft whose jobs finished today and store it in completed; later checks/updates use this named intermediate.

489 Store the result in grounded so later lines can use it.

490 Loop through sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i)), handling each aircraft once.

491 Check for an existing committed job before deciding whether the aircraft can receive a new plan.

492 Skip this item and begin the next loop iteration.

493 Compute the validation or resource-conflict reason and store it in reason; later checks/updates use this named intermediate.

494 When reason is None and int(a[aircraft, 0]) is true, enter this block; otherwise continue to the next case.

495 Separate this result into reason, cost for the steps that follow.

496 Add cost to the running total in costs.

When reason is true, enter this block; otherwise continue to the next case.

Add this item to rejected so it stays available for later use.

Add 2.0 to the running total in costs.

Check for an existing committed job before deciding whether the aircraft can receive a new plan.

Call grounded.add to perform this step of the surrounding algorithm.

Add float(not int(a[aircraft, 0])) to the running total in self.metrics['unsafe_attempts'].

Check for an existing committed job before deciding whether the aircraft can receive a new plan.

Call grounded.add to perform this step of the surrounding algorithm.

Add len(rejected) to the running total in self.metrics['rejections'].

Compute the number of aircraft that flew today and store it in flights; later checks/updates use this named intermediate.

Compute the number of failures in this transition or episode and store it in failures; later checks/updates use this named intermediate.

Compute whether the shared weather event occurs today and store it in weather; later checks/updates use this named intermediate.

Loop through range(cfg.aircraft), handling each aircraft once.

When aircraft in grounded is true, enter this block; otherwise continue to the next case.

Skip this item and begin the next loop iteration.

Add 1 to the running total in flights.

Subtract WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0, self.wear_tape[self.day, aircraft]) * (3 if weather else 1) from self.health[aircraft] to account for work or wear during this transition.

When weather is true, enter this block; otherwise continue to the next case.

Add 1 to the running total in self.metrics['weather_exposures'].

Loop through range(2), handling each e once.

Loop through [('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]), ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])], handling each kind, p, impact once.

Store the result in k so later lines can use it.

When self.event_tape[self.day, aircraft, e, k] < p is true, enter this block; otherwise continue to the next case.

Store the result in severity so later lines can use it.

Subtract severity * np.array(impact) from self.health[aircraft, e] to account for work or wear during this transition.

Add this item to events so it stays available for later use.

Add 1 to the running total in self.metrics[kind + '_events'].

Separate this result into np.clip(self.health[aircraft], 0, 1, out for the steps that follow.

Store the result in failed so later lines can use it.

Add failed to the running total in failures.

Update self.measurements[aircraft]. it holds noisy sensor readings, not hidden true health for later transitions.

Add this aircraft's generated sensor noise to its efficiency and temperature readings.

Call self._clip_measurements to keep sensor values inside the supported ranges.

Store this value as self.last_flight_day[aircraft] on the object so later method calls can read it.

Call self._update_filter to update health particles from the new sensor measurements.

Loop through list(self.jobs.items()), handling each aircraft, j once.

Convert total duration minus remaining days into a one-based elapsed-day count for locating the current work stage.

Begin the repair schedule after the fixed setup delay; no component task can finish before setup is complete.

Loop through j.tasks, handling each e, c once.

Add this component's labor time divided across assigned crews, rounding up because a fractional workday still consumes a whole day.

When elapsed == task_day is true, enter this block; otherwise continue to the next case.

When the scheduled task completes, set the replaced component's hidden health to 1.0, the simulator's fully-new value.

Reset the belief particles for the replaced component to fully healthy because the repair outcome is known.

Increment the completed-component replacement counter once for this task.

Subtract 1 from j.remaining to account for work or wear during this transition.

When j.remaining == 0 is true, enter this block; otherwise continue to the next case.

Add this item to completed so it stays available for later use.

Continue step; this expression contributes to its current state update or result.

Store the result in downtime so later lines can use it.

Add downtime * 2 + failures * 80 to the running total in costs.

Calculate this step's reward and store it for the transition result and episode return.

Add reward to the running total in self.metrics['return'].

Add costs to the running total in self.metrics['cost'].

Add flights to the running total in self.metrics['flights'].

Add failures to the running total in self.metrics['flight_failures'].

552 Add downtime to the running total in self.metrics['downtime_aircraft_days'].

553 Add 1 to the running total in self.day.

554 When self.day % cfg.replenish_days == 0 is true, enter this block; otherwise continue to the next case.

555 Update self.stock. it holds the spare-part inventory by component type for later transitions.

556 Store this value as self._done on the object so later method calls can read it.

557 Store the result in info so later lines can use it.

558 Return the current observation, reward, terminal flags, and info in the API's documented order.

8.5.16 `FleetMaintenanceEnv.render`

This returns a short human-readable snapshot and calls resource_state() so displayed crew and workshop availability matches active jobs.

```
560     def render(self):
561         return f'day={self.day} busy={sorted(self.jobs)} crews/shops={self.resource_state()}
        ↪  stock={self.stock.tolist()}'
```

Line-by-line explanation.

560 Define render with the shown parameters/defaults; its purpose is to format simulator state for a human.

561 Return a readable summary of the current day, health, remaining time, jobs, resources, and stock.

8.6. Function `threshold_action`

This policy is a hand-written baseline. evaluate can compare it with trained policies, and execute_option uses it to turn fixed maintenance options into plans when no learned worker plan is supplied.

```
563 def threshold_action(env, observation, threshold=0.65, bundle_threshold=0.75):
564     """Feasible rule using only observations, not hidden health or event diagnostics."""
565     rows, public = env.split_observation(observation)
566     action = env.idle_action()
567     free = int(round(public[1] * env.config.crews))
568     used = {int(round(r[43] * max(1, env.config.workshops - 1))) for r in rows if r[37] > 0.5}
569     shops = [s for s in range(env.config.workshops) if s not in used]
570     stock = np.rint(public[4:] * env.config.aircraft).astype(int)
571     scores = rows[:, 24:36].min(1)
572     rank = np.argsort(scores)
573     for priority, aircraft in enumerate(rank[::-1]):
574         action[aircraft, 7] = priority
575     for aircraft in rank:
576         if rows[aircraft, 37] > 0.5 or scores[aircraft] > threshold or free < 1 or (not shops):
577             continue
578         health = rows[aircraft, :12].reshape(2, 6)
579         selected = health <= bundle_threshold
580         if not selected.any():
581             selected.flat[rows[aircraft, 24:36].argmin()] = True
582         bundles = [0, 0]
583         for e in range(2):
584             for c in range(6):
585                 if selected[e, c] and stock[c] > 0:
```

```
586                     bundles[e] |= 1 << c
587                     stock[c] -= 1
588         mask = int(bundles[0] != 0) + 2 * int(bundles[1] != 0)
589         if not mask:
590             continue
591         action[aircraft] = [1, mask, *bundles, 0, shops.pop(0), int(health.argmin()), action[aircraft,
            ↪  7]]
592         free -= 1
593     return action
```

Line-by-line explanation.

563 Define threshold_action with the shown parameters/defaults; its purpose is to build a fleet plan from observed estimates while respecting busy aircraft and public resources.

564 This docstring states threshold_action's purpose/contract: Feasible rule using only observations, not hidden health or event diagnostics. It is documentation text, not an executable operation.

565 Separate this result into rows, public for the steps that follow.

566 Store the proposed action in action for the following checks and updates.

567 Compute the number of crews not committed to jobs and store it in free; later checks/updates use this named intermediate.

568 Store the result in used so later lines can use it.

569 Compute the currently available workshops and store it in shops; later checks/updates use this named intermediate.

570 Compute the spare-part inventory by component type and store it in stock; later checks/updates use this named intermediate.

571 Compute the candidate action or option values being compared and store it in scores; later checks/updates use this named intermediate.

572 Store the result in rank so later lines can use it.

573 Loop through enumerate(rank[::-1]), handling each priority, aircraft once.

574 Separate this result into action[aircraft, 7] for the steps that follow.

575 Loop through rank, handling each aircraft once.

576 When rows[aircraft, 37] $>$ 0.5 or scores[aircraft] $>$ threshold or free $<$ 1 or (not shops) is true, enter this block; otherwise continue to the next case.

577 Skip this item and begin the next loop iteration.

578 Reshape this aircraft's first 12 observation values into [engine, component]. They are policy-visible estimates, not hidden simulator truth.

579 Build a 2×6 Boolean grid selecting each component whose estimated mean health is at or below the bundle threshold.

580 When not selected.any() is true, enter this block; otherwise continue to the next case.

581 If the mean-health threshold selected nothing, flatten the 2×6 grid and force its weakest lower-health estimate to True. This fallback asks for at least one repair, if stock permits.

582 Compute the separate six-bit component masks, one per fleet engine and store it in bundles; later checks/updates use this named intermediate.

583 Loop through range(2), handling each e once.

584 Loop through range(6), handling each c once.

585 When selected[e, c] and stock[c] > 0 is true, enter this block; otherwise continue to the next case.

586 Set component c in engine e's bundle without erasing bits selected earlier. For the ordered components fan/LPC/HPC/combustor/HPT/LPT, shifts 1, 2, 4, 8, 16, 32 represent indices 0–5. For example, 5 is 000101_2 and selects fan plus HPC. |= combines flags, not component numbers. Bundle zero means no new work for that engine; nonzero means at least one component is requested.

587 Reserve one spare of component type c for this proposed task so later aircraft in priority order cannot allocate the same inventory unit.

588 Turn two nonempty-bundle tests into engine flags: engine 0 contributes bit 0/value 1; engine 1 contributes bit 1/value 2. [5,0]→01_2 (engine 0 only), [0,8]→10_2 (engine 1 only), [5,8]→11_2 (both). Each bundle is its own six-bit component mask; this two-bit mask only records which engines have work.

589 When not mask is true, enter this block; otherwise continue to the next case.

590 Skip this item and begin the next loop iteration.

591 Write the row in its eight-field order: ground, engine mask, two bundles, crew index 0 (one crew), assigned shop, weakest estimated-health component as order start, and this aircraft's existing priority.

592 Commit one crew to the newly accepted job; fewer crews remain for lower-priority aircraft.

593 Return action to the caller; this ends the function and supplies its result.

8.7. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
594
595
596 # ====================================================================
```

Line-by-line explanation.

596 This comment draws a visual boundary between source sections. It has no runtime effect.

9. The action-shape adapter

The simulator uses a matrix with one eight-field action row per aircraft. FlattenFleetAction presents the same discrete choices as one flat vector for libraries that require a one-dimensional MultiDiscrete action space, then reshapes actions before they reach the environment.

The wrapper changes only representation. A flattened vector is reshaped back to the matrix expected by FleetMaintenanceEnv; reverse_action performs the opposite mapping. It does not alter action meanings, feasibility, rewards, or dynamics.

9.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
# fleet_maintenance/adapters.py
# =============================================================================


import numpy as np
import gymnasium as gym
from gymnasium import spaces

```

Line-by-line explanation.

597 This source comment labels or clarifies the section; Python does not execute it.

598 This comment draws a visual boundary between source sections. It has no runtime effect.

601 Use NumPy for arrays, vectorized simulation, random sampling, and numeric calculations.

602 Use Gymnasium environment/wrapper base classes under the conventional alias.

603 Declare and validate legal observation and action shapes.

9.2. Class `FlattenFleetAction`

This class an adapter that flattens fleet actions for Stable-Baselines3 and restores them before simulation.

```
class FlattenFleetAction(gym.ActionWrapper):
    """Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec."""

```

Line-by-line explanation.

605 Define class FlattenFleetAction: an adapter that flattens fleet actions for Stable-Baselines3 and restores them before simulation.

606 This docstring states FlattenFleetAction's purpose/contract: Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec. It is documentation text, not an executable operation.

9.2.1 `FlattenFleetAction.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
    def __init__(self, env):
        super().__init__(env)
        self.action_shape = env.action_space.nvec.shape
        self.action_space = spaces.MultiDiscrete(env.action_space.nvec.ravel())
```

Line-by-line explanation.

608 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

609 Initialize the parent class before setting subclass-specific state.

610 Store this value as self.action_shape on the object so later method calls can read it.

611 Store this value as self.action_space on the object so later method calls can read it.

9.2.2 `FlattenFleetAction.action`

Gymnasium calls this wrapper when an outer algorithm sends a flat action; it restores the simulator's matrix shape before delegation.

```
613     def action(self, action):
614         return np.asarray(action, dtype=np.int64).reshape(self.action_shape)
```

Line-by-line explanation.

613 Define action with the shown parameters/defaults; its purpose is to reshape a flat action into the simulator's matrix format.

614 Return the value produced by this expression to the caller.

9.2.3 `FlattenFleetAction.reverse_action`

This method flatten a matrix action for the outer library.

```
616     def reverse_action(self, action):
617         return np.asarray(action, dtype=np.int64).ravel()
```

Line-by-line explanation.

616 Define reverse_action with the shown parameters/defaults; its purpose is to flatten a matrix action for the outer library.

617 Return the value produced by this expression to the caller.

9.3. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
618
619
620 # ====================================================================
```

Line-by-line explanation.

620 This comment draws a visual boundary between source sections. It has no runtime effect.

10. The actor-critic policies and option executor

This module defines the learned decision rules. FleetActorCritic builds per-aircraft action distributions and a fleet-level value estimate. OptionActorCritic selects among four manager actions. LearnedOptionActorCritic combines a manager with a goal-conditioned worker. execute_option keeps an accepted plan committed while the simulated fleet advances for multiple days. The code calls these choices options, but its docstring says the learned macros are latent planning contexts, not certified skills or a canonical Option-Critic implementation.

PyTorch's nn.Module is the base class that registers trainable parameters and lets the optimizer discover them. nn.Linear implements learned affine maps, nn.Sequential composes layers, and nn.ModuleList registers a variable collection of action heads and conditioning projections. The global_encoder supplies shared fleet context to actor features; the critic is a separate network that estimates V(s). Categorical represents each discrete field's probabilities, allowing the policy to sample actions and compute log-probability and entropy. In training, act() is first called without actions, so the policy samples. During the PPO update, the stored action is supplied so the current policy can score that same action again.

10.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
621 # fleet_maintenance/policies.py
622 # ======================================================================
623
624
625 import numpy as np
626 import torch
627 from torch import nn
628 from torch.distributions import Categorical
629
```

Line-by-line explanation.

621 This source comment labels or clarifies the section; Python does not execute it.

622 This comment draws a visual boundary between source sections. It has no runtime effect.

625 Use NumPy for arrays, vectorized simulation, random sampling, and numeric calculations.

626 Use tensors, neural networks, optimizers, and automatic differentiation.

627 Use PyTorch's module and layer classes to construct actor/critic networks.

628 Represent a discrete choice distribution with sampling, log-probability, and entropy.

10.2. Class `FleetActorCritic`

This class a constrained action policy plus a critic that estimates state value.

```
630 class FleetActorCritic(nn.Module):
631     """Central critic + flat, conditional, or parameter-shared aircraft actors.
632
633     shared: local aircraft estimates + public resource counts. Parameters reused
634     across aircraft; a central deterministic resolver enforces resource capacity.
635     conditional: global actor context and masked dependent action distributions.
```

```
636     flat: shared global features with no learned conditioning on preceding choices.
637     All methods use the same logical masks; scarce-resource conflicts are resolved
638     by the simulator and reported separately from structural feasibility.
639     All act daily and obey pre-start commitment; this is not temporal HRL.
640     """
641
```

Line-by-line explanation.

630 Define class FleetActorCritic: a constrained action policy plus a critic that estimates state value.

631 This docstring states FleetActorCritic's purpose/contract: Central critic + flat, conditional, or parameter-shared aircraft actors. It is documentation text, not an executable operation.

633 This docstring line states: shared: local aircraft estimates + public resource counts. Parameters reused

634 This docstring line states: across aircraft; a central deterministic resolver enforces resource capacity.

635 This docstring line states: conditional: global actor context and masked dependent action distributions.

636 This docstring line states: flat: shared global features with no learned conditioning on preceding choices.

637 This docstring line states: All methods use the same logical masks; scarce-resource conflicts are resolved

638 This docstring line states: by the simulator and reported separately from structural feasibility.

639 This docstring line states: All act daily and obey pre-start commitment; this is not temporal HRL.

640 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

10.2.1 `FleetActorCritic.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
642     def __init__(self, env: FleetMaintenanceEnv, mode='conditional', goal_size=0):
643         super().__init__()
644         if mode not in ('flat', 'conditional', 'shared'):
645             raise ValueError(mode)
646         self.mode = mode
647         self.n = env.config.aircraft
648         self.local = env.local_size
649         self.public = env.public_size
650         self.dims = env.row_dims.tolist()
651         self.mandatory = env.config.mandatory_health
652         self.goal_size = goal_size
653         size = env.observation_space.shape[0]
654         self.global_encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh())
655         in_size = self.local + self.public + (0 if mode == 'shared' else 64) + goal_size
656         self.actor = nn.Sequential(nn.Linear(in_size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
657         self.heads = nn.ModuleList((nn.Linear(64, d) for d in self.dims))
658         self.condition = nn.ModuleList((nn.Linear(8, 64, bias=False) for _ in self.dims))
659         self.critic = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(),
        ↪  nn.Linear(64, 1))
```

Line-by-line explanation.

642 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

643 Initialize the parent class before setting subclass-specific state.

644 When mode not in ('flat', 'conditional', 'shared') is true, enter this block; otherwise continue to the next case.

645 Raise ValueError(mode); invalid input stops here instead of creating inconsistent state.

646 Store this value as self.mode on the object so later method calls can read it.

647 Store this value as self.n on the object so later method calls can read it.

648 Compute each aircraft's own observation slice and store it in self.local; later checks/updates use this named intermediate.

649 Compute fleet-wide observation values shared by aircraft and store it in self.public; later checks/updates use this named intermediate.

650 Store this value as self.dims on the object so later method calls can read it.

651 Compute whether a lower-health safety alarm is active and store it in self.mandatory; later checks/updates use this named intermediate.

652 Store this value as self.goal_size on the object so later method calls can read it.

653 Store the result in size so later lines can use it.

654 Build a shared 64-unit encoder for the full fleet observation. It converts the global state into a compact context vector; Tanh bounds each encoded feature.

655 Calculate the per-aircraft actor input width: local row + broadcast public fleet data + optional 64-feature global context (omitted in shared mode) + optional worker goal.

656 Build the actor's two-layer feature network. It maps each aircraft's assembled inputs to a 64-number representation used by every action-field head.

657 Create one output head per action field; each emits as many logits as that field has categories. ModuleList registers all heads so PyTorch trains/saves their parameters.

658 Create one conditioning projection per action field. It maps the eight previously selected field values into the current 64-feature state; bias=False keeps this as a pure learned interaction.

659 Build the critic on the complete fleet observation; the final single output is V(s), the estimated return from this state.

10.2.2 `FleetActorCritic.value`

The training loop uses this critic estimate for value bootstrapping and the value-loss target. It is not the action selector.

```
661     def value(self, obs):
662         return self.critic(obs).squeeze(-1)
```

Line-by-line explanation.

661 Define value with the shown parameters/defaults; its purpose is to run the critic to estimate the current state's value.

662 Return self.critic(obs).squeeze(-1) to the caller; this ends the function and supplies its result.

10.2.3 `FleetActorCritic.act`

The actor uses the current observation to choose or rescore an action. It returns the action, joint log-probability, entropy, and critic value. Conditional mode chooses action fields in order and masks invalid choices; the simulator still resolves shared resources and reports rejected plans.

```
664     def act(self, obs, actions=None, deterministic=False, goal=None):
665         if obs.ndim == 1:
666             obs = obs.unsqueeze(0)
667         batch = obs.shape[0]
668         local = obs[:, :-self.public].reshape(batch, self.n, self.local)
669         public = obs[:, -self.public:].unsqueeze(1).expand(-1, self.n, -1)
670         features = [local, public]
671         if self.goal_size:
672             if goal is None:
673                 raise ValueError('goal-conditioned worker requires a goal')
674             features.append(goal.unsqueeze(1).expand(-1, self.n, -1))
675         if self.mode != 'shared':
676             features.append(self.global_encoder(obs).unsqueeze(1).expand(-1, self.n, -1))
677         h = self.actor(torch.cat(features, -1))
678         previous = torch.zeros(batch, self.n, 8, device=obs.device)
679         logprob = torch.zeros(batch, device=obs.device)
680         entropy = torch.zeros_like(logprob)
681         result = []
682         busy = local[:, :, 37] > 0.5
683         if self.goal_size:
684             busy = busy | (goal[:, 0] > 0.5).unsqueeze(-1)
685         urgent = local[:, :, 24:36].amin(-1) <= self.mandatory
686         for k, head in enumerate(self.heads):
687             logits = head(h if self.mode == 'flat' else torch.tanh(h + self.condition[k](previous)))
688             valid = torch.ones_like(logits, dtype=torch.bool)
689             valid[busy] = False
690             valid[:, :, 0] |= busy
691             if k == 0:
692                 valid[:, :, 0] &= ~(urgent & ~busy)
693             else:
694                 ground = result[0] == 1
695                 if k == 1:
696                     valid[:, :, 0] &= ~ground
697                 if k in (2, 3):
698                     engine_on = result[1] & 1 << k - 2 != 0
699                     active = ground & engine_on & ~busy
700                     valid[:, :, 0] &= ~active
701                     valid[:, :, 1:] &= active.unsqueeze(-1)
702                 elif k >= 4:
703                     valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
704                 if k == 1:
705                     valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
706             dist = Categorical(logits=logits.masked_fill(~valid, -1000000000.0))
707             choice = actions[:, :, k] if actions is not None else dist.probs.argmax(-1) if
                ↪  deterministic else dist.sample()
708             result.append(choice)
709             logprob += dist.log_prob(choice).sum(-1)
710             entropy += dist.entropy().sum(-1)
711             addition = torch.zeros_like(previous)
712             addition[:, :, k] = choice / float(max(1, self.dims[k] - 1))
713             previous = previous + addition
714         return (torch.stack(result, -1), logprob, entropy, self.value(obs))
```

Line-by-line explanation.

Define act with the shown parameters/defaults; its purpose is to choose an action or calculate statistics for a supplied action.

When obs.ndim == 1 is true, enter this block; otherwise continue to the next case.

Update obs to the current encoded state before the next decision.

Store the result in batch so later lines can use it.

Compute each aircraft's own observation slice and store it in local; later checks/updates use this named intermediate.

Compute fleet-wide observation values shared by aircraft and store it in public; later checks/updates use this named intermediate.

Start each aircraft's actor input with its own local row and the shared public-resource vector; both are needed to choose a feasible plan.

When self.goal_size is true, enter this block; otherwise continue to the next case.

When goal is None is true, enter this block; otherwise continue to the next case.

Raise ValueError('goal-conditioned worker requires a goal'); invalid input stops here instead of creating inconsistent state.

Broadcast the manager's one-hot goal across all aircraft so every worker decision is conditioned on the same selected macro.

When self.mode != 'shared' is true, enter this block; otherwise continue to the next case.

Encode the whole fleet once, add an aircraft axis, then broadcast the same global context to each aircraft's actor input.

Concatenate each aircraft's local/public/context/goal features and pass them through the actor MLP to get per-aircraft hidden features.

Initialize the eight action-field conditioning slots to zero; as fields are chosen, their normalized values are filled in for later heads.

Start one joint log-probability accumulator per observation in the batch; each chosen field contributes to it.

Start an entropy accumulator with the same batch shape/device as logprob.

Store the result in result so later lines can use it.

Read the busy indicator from local observation column 37 for every batch item and aircraft; busy aircraft already have a committed plan.

When self.goal_size is true, enter this block; otherwise continue to the next case.

Compute whether an aircraft already has a committed job and store it in busy; later checks/updates use this named intermediate.

Take the minimum of the 12 per-component lower-health estimates in columns 24–35; mark the aircraft urgent if any estimate crosses the mandatory threshold.

Loop through enumerate(self.heads), handling each k, head once.

Produce this field's category scores. Flat mode uses h directly; conditional/shared modes add a learned projection of earlier choices and pass the sum through Tanh.

Initially allow every category for every aircraft; subsequent mask rules remove choices that violate action dependencies or safety constraints.

689 Temporarily invalidate every category for busy aircraft; their committed job must not be overwritten by a new plan.

690 Restore the no-op choice for busy aircraft; their current job continues and this action cannot assign a second plan.

691 When k == 0 is true, enter this block; otherwise continue to the next case.

692 Forbid the no-grounding choice for urgent aircraft that are not already busy; a running job is not replanned.

693 Otherwise, follow this alternative branch.

694 Read the already-selected first field and convert it to a Boolean: this aircraft was chosen for grounding/maintenance.

695 When k == 1 is true, enter this block; otherwise continue to the next case.

696 For the engine-mask field, disallow engine mask zero when grounding was selected; a grounded aircraft needs at least one engine with a nonempty bundle.

697 When k in (2, 3) is true, enter this block; otherwise continue to the next case.

698 For bundle heads k=2/3, select bit 0/1 from the chosen engine mask. Shift 1 by k-2, AND it with result[1], then compare with zero. Python evaluates shifts before &, so this is result[1] & (1 « (k-2)) != 0.

699 Compute whether a selected engine may receive a new component bundle and store it in active; later checks/updates use this named intermediate.

700 Forbid bundle zero on an active engine: once selected for work, an engine must get a nonempty component bundle.

701 Allow nonzero bundles only on active engines. The final axis is the bundle-choice axis; unsqueeze adds a broadcast dimension so each active flag gates every nonzero bundle.

702 When k >= 4 is true, enter this block; otherwise continue to the next case.

703 For the current engine/resource field, permit nonzero categories only for a new plan that is grounded and not busy; unsqueeze broadcasts that eligibility flag across the category axis.

704 When k == 1 is true, enter this block; otherwise continue to the next case.

705 For the current engine/resource field, permit nonzero categories only for a new plan that is grounded and not busy; unsqueeze broadcasts that eligibility flag across the category axis.

706 Give invalid choices an extremely negative logit, making their softmax probability effectively zero before constructing the categorical distribution.

707 When training, reuse the stored action to recompute its likelihood. Otherwise take argmax for deterministic evaluation or sample a categorical choice for exploration.

708 Add this item to result so it stays available for later use.

709 Accumulate the selected field's log-probability across aircraft; PPO uses the joint action likelihood in its policy ratio.

710 Accumulate distribution entropy across aircraft; its bonus encourages the policy to retain exploration.

711 Create a zero tensor shaped like the prior-field buffer so only this action coordinate will be written before accumulation.

712 Store this selected category in slot k, scaled into roughly 0–1; normalization lets later conditioning layers consume fields with different category counts.

713 Add the newly selected normalized field to the conditioning buffer without changing earlier field choices.

714 Return the selected fields stacked on the last axis, their joint log-probability, summed entropy, and critic value; these outputs feed rollout storage and PPO.

10.3. Class `OptionActorCritic`

This class a manager policy that chooses one of four high-level options and estimates value.

```
716 class OptionActorCritic(nn.Module):
717     """Learned manager selects one of four fixed fleet planning options."""
718 
```

Line-by-line explanation.

716 Define class OptionActorCritic: a manager policy that chooses one of four high-level options and estimates value.

717 This docstring states OptionActorCritic's purpose/contract: Learned manager selects one of four fixed fleet planning options. It is documentation text, not an executable operation.

10.3.1 `OptionActorCritic.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
719     def __init__(self, env):
720         super().__init__()
721         size = env.observation_space.shape[0]
722         self.encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
723         self.actor = nn.Linear(64, 4)
724         self.critic = nn.Linear(64, 1)
```

Line-by-line explanation.

719 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

720 Initialize the parent class before setting subclass-specific state.

721 Store the result in size so later lines can use it.

722 Encode the whole fleet observation into a 64-feature manager representation using two Linear/Tanh stages.

723 Map the manager representation to four option logits, one for each high-level option ID.

724 Map the same manager representation to one scalar state-value estimate.

10.3.2 `OptionActorCritic.value`

This method run the critic to estimate the current state's value.

```
726     def value(self, obs):
727         return self.critic(self.encoder(obs)).squeeze(-1)
```

Line-by-line explanation.

726 Define value with the shown parameters/defaults; its purpose is to run the critic to estimate the current state's value.

727 Return the value produced by this expression to the caller.

10.3.3 `OptionActorCritic.act`

This manager's action is an integer from zero through three. In train, execute_option interprets that integer as wait, targeted maintenance, opportunistic bundle, or preventive bundle.

```
729     def act(self, obs, actions=None, deterministic=False):
730         if obs.ndim == 1:
731             obs = obs.unsqueeze(0)
732         h = self.encoder(obs)
733         dist = Categorical(logits=self.actor(h))
734         action = actions if actions is not None else dist.probs.argmax(-1) if deterministic else
            ↪  dist.sample()
735         return (action, dist.log_prob(action), dist.entropy(), self.critic(h).squeeze(-1))
```

Line-by-line explanation.

729 Define act with the shown parameters/defaults; its purpose is to choose an action or calculate statistics for a supplied action.

730 When obs.ndim == 1 is true, enter this block; otherwise continue to the next case.

731 Update obs to the current encoded state before the next decision.

732 Compute the manager's learned feature representation of the current fleet state.

733 Convert the four manager logits into a categorical option distribution; softmax probabilities are implicit.

734 Store the proposed action in action for the following checks and updates.

735 Return these values together; callers unpack them in the order shown.

10.4. Class `LearnedOptionActorCritic`

This class a jointly trained manager and goal-conditioned worker that outputs a macro plus fleet plan.

```
737 class LearnedOptionActorCritic(nn.Module):
738     """Jointly trained manager + goal-conditioned plan worker + fixed executor.
739 
740     Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.
741     The worker selects engines, components and resources once before execution.
742     Both levels learn from fleet return. This is not MAXQ or option-critic and
743     option collapse/credit assignment require later ablations and diagnostics.
744     """
745 
```

Line-by-line explanation.

737 Define class LearnedOptionActorCritic: a jointly trained manager and goal-conditioned worker that outputs a macro plus fleet plan.

738 This docstring states LearnedOptionActorCritic's purpose/contract: Jointly trained manager + goal-conditioned plan worker + fixed executor. It is documentation text, not an executable operation.

740 This docstring line states: Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.

741 This docstring line states: The worker selects engines, components and resources once before execution.

742 This docstring line states: Both levels learn from fleet return. This is not MAXQ or option-critic and

743 This docstring line states: option collapse/credit assignment require later ablations and diagnostics.

744 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

10.4.1 `LearnedOptionActorCritic.__init__`

This method initialize instance state, spaces, configuration, or neural-network layers.

```
746     def __init__(self, env):
747         super().__init__()
748         self.manager = OptionActorCritic(env)
749         self.worker = FleetActorCritic(env, 'conditional', goal_size=4)
750         self.n = env.config.aircraft
```

Line-by-line explanation.

746 Define ___init___ with the shown parameters/defaults; its purpose is to initialize instance state, spaces, configuration, or neural-network layers.

747 Initialize the parent class before setting subclass-specific state.

748 Store this value as self.manager on the object so later method calls can read it.

749 Store this value as self.worker on the object so later method calls can read it.

750 Store this value as self.n on the object so later method calls can read it.

10.4.2 `LearnedOptionActorCritic.value`

This method run the critic to estimate the current state's value.

```
752     def value(self, obs):
753         return self.manager.value(obs)
```

Line-by-line explanation.

752 Define value with the shown parameters/defaults; its purpose is to run the critic to estimate the current state's value.

753 Return self.manager.value(obs) to the caller; this ends the function and supplies its result.

10.4.3 `LearnedOptionActorCritic.act`

This first samples or rescales a manager option, converts the option to a one-hot goal, and calls the worker to produce a fleet plan. The composite action lets the trainer store and later rescore both decisions.

```
755     def act(self, obs, actions=None, deterministic=False):
756         if obs.ndim == 1:
757             obs = obs.unsqueeze(0)
758         macro = None if actions is None else actions[:, 0]
759         plan = None if actions is None else actions[:, 1:].reshape(-1, self.n, 8)
760         option, lp, entropy, value = self.manager.act(obs, macro, deterministic)
761         goal = torch.nn.functional.one_hot(option, num_classes=4).float()
762         plan, worker_lp, worker_entropy, _ = self.worker.act(obs, plan, deterministic, goal)
763         composite = torch.cat([option.unsqueeze(-1), plan.flatten(1)], -1)
764         return (composite, lp + worker_lp, entropy + worker_entropy, value)
```

Line-by-line explanation.

755 Define act with the shown parameters/defaults; its purpose is to choose an action or calculate statistics for a supplied action.

756 When obs.ndim == 1 is true, enter this block; otherwise continue to the next case.

757 Update obs to the current encoded state before the next decision.

758 Compute the manager's high-level option ID and store it in macro; later checks/updates use this named intermediate.

759 Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

760 Receive the action, log-probability, entropy, and value. The underscores discard outputs this call does not need.

761 Represent each selected option ID as a four-element one-hot goal, which becomes the worker's conditioning input.

762 Receive the action, log-probability, entropy, and value. The underscores discard outputs this call does not need.

763 Flatten each worker plan to one row, then prepend its manager option ID so the stored composite action can later be split and rescored.

764 Return macro-plus-plan, joint manager/worker log-probability, joint entropy, and manager value; the two policy levels are trained from the same return.

10.5. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
765 OPTION_NAMES = ('wait_one_day', 'targeted_maintenance', 'opportunistic_bundle', 'preventive_bundle')
766
```

Line-by-line explanation.

765 Define shared module constant OPTION_NAMES; other examples reuse this value.

10.6. Function execute_option

train and evaluate call this after the manager chooses a macro. The executor applies one plan and then keeps existing jobs committed while other aircraft continue operating, until completion, new observed urgency, episode end, or the day limit.

```
767 def execute_option(env, option, obs, gamma=0.99, max_days=14, planned_action=None):
768     """Plan once, then keep all active jobs committed while the fleet advances.
769 
770     Manager returns at first task completion, a new observed urgency alarm,
771     episode end, or max_days. It never consumes true event diagnostics.
772     Waiting without any accepted job lasts one day.
773     Other operating aircraft continue their modeled flights during this option.
774     Longer manager intervals change decision opportunities, so this is a timing
775     experiment as well as an architecture experiment, not a matched PPO ablation.
776     """
777     option = int(option)
778     if option not in range(4):
779         raise ValueError('unknown option')
780     if max_days < 1:
781         raise ValueError('max_days must be positive')
782     thresholds = {1: (0.62, 0.62), 2: (0.65, 0.77), 3: (0.78, 0.85)}
783     action = planned_action if planned_action is not None else env.idle_action() if option == 0 else
        ↪  threshold_action(env, obs, *thresholds[option])
784     before, _ = env.split_observation(obs)
785     before_alarm = before[:, 24:36].min(1) <= env.config.mandatory_health
786     rewards = []
787     while True:
788         obs, reward, done, truncated, info = env.step(action)
789         rewards.append(reward)
790         rows, _ = env.split_observation(obs)
791         new_alarm = np.any((rows[:, 24:36].min(1) <= env.config.mandatory_health) & ~before_alarm)
792         if option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >=
            ↪  max_days) or (not env.jobs):
793             break
794         action = env.idle_action()
795     discounted = sum((gamma ** i * r for i, r in enumerate(rewards)))
796     return (obs, discounted, done, truncated, info, len(rewards), sum(rewards))
```

Line-by-line explanation.

767 Define execute_option with the shown parameters/defaults; its purpose is to execute a fleet option for several days while accepted jobs remain committed.

768 This docstring states execute_option's purpose/contract: Plan once, then keep all active jobs committed while the fleet advances. It is documentation text, not an executable operation.

770 This docstring line states: Manager returns at first task completion, a new observed urgency alarm,

771 This docstring line states: episode end, or max_days. It never consumes true event diagnostics.

772 This docstring line states: Waiting without any accepted job lasts one day.

773 This docstring line states: Other operating aircraft continue their modeled flights during this option.

774 This docstring line states: Longer manager intervals change decision opportunities, so this is a timing

775 This docstring line states: experiment as well as an architecture experiment, not a matched PPO ablation.

776 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

777 Compute the selected high-level action and store it in option; later checks/updates use this named intermediate.

778 When option not in range(4) is true, enter this block; otherwise continue to the next case.

779 Raise ValueError('unknown option'); invalid input stops here instead of creating inconsistent state.

780 When max_days < 1 is true, enter this block; otherwise continue to the next case.

781 Raise ValueError('max_days must be positive'); invalid input stops here instead of creating inconsistent state.

782 Store the result in thresholds so later lines can use it.

783 Store the proposed action in action for the following checks and updates.

784 Unpack the return values from env.split_observation(obs) into before; underscore entries are intentionally ignored.

785 Compute alarms already present before option execution and store it in before_alarm; later checks/updates use this named intermediate.

786 Compute the primitive rewards collected during an option and store it in rewards; later checks/updates use this named intermediate.

787 Continue the loop until a stopping condition triggers break or return.

788 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

789 Add this item to rewards so it stays available for later use.

790 Unpack the return values from env.split_observation(obs) into rows; underscore entries are intentionally ignored.

791 Compute a newly appearing alarm that can end option execution and store it in new_alarm; later checks/updates use this named intermediate.

792 Check the episode/option stop condition option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >= max_days) or (not env.jobs); when true, stop collecting transitions for this segment.

793 Stop this loop because its exit condition is satisfied.

794 Store the proposed action in action for the following checks and updates.

795 Compute the option return after applying time-discount factors and store it in discounted; later checks/updates use this named intermediate.

796 Return these values together; callers unpack them in the order shown.

10.7. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
797
798
799 # ====================================================================
```

Line-by-line explanation.

799 This comment draws a visual boundary between source sections. It has no runtime effect.

11. Training, checkpointing, and evaluation

The trainer constructs a selected policy, samples actions, advances the fixed simulator, stores transitions, computes duration-aware advantages and return targets, and updates neural-network parameters with a PPO-style objective. evaluate runs fixed-seed episodes without updating parameters. main parses command-line arguments and writes the requested experiment outputs.

The function train learns the policy and critic parameters, not the simulator. The environment's transition rules and parameters remain fixed while the model interacts with it. When method is flat, conditional, or shared, each policy decision advances one day. With options, a learned manager chooses a fixed planning rule. With learned_options, the manager and goal-conditioned worker both choose parts of the plan. execute_option then advances the environment until a stopping condition. Such decisions can span different numbers of days, so smdp_gae discounts rewards and bootstrapped values by duration. This changes decision timing as well as architecture, which the code's own docstring flags as a comparison limitation. Evaluation uses deterministic model actions or the threshold baseline and does not update weights.

11.1. Module imports and declarations

This block establishes the module's imports, constants, or public interface.

```
# fleet_maintenance/train.py
# ============================================================================


from dataclasses import asdict
from pathlib import Path
import argparse
import json
import numpy as np
import torch

```

Line-by-line explanation.

800 This source comment labels or clarifies the section; Python does not execute it.

801 This comment draws a visual boundary between source sections. It has no runtime effect.

804 Convert configuration records to dictionaries for saving experiment metadata.

805 Construct output paths and create directories portably.

806 Parse command-line settings for training and evaluation.

807 Serialize experiment results in a portable text format.

808 Use NumPy for arrays, vectorized simulation, random sampling, and numeric calculations.

809 Use tensors, neural networks, optimizers, and automatic differentiation.

11.2. Function `make_model`

This function select the neural architecture named by the training method.

```
811 def make_model(env, method):
812     if method == 'learned_options':
813         return LearnedOptionActorCritic(env)
814     return OptionActorCritic(env) if method == 'options' else FleetActorCritic(env, method)
```

Line-by-line explanation.

811 Define make_model with the shown parameters/defaults; its purpose is to select the neural architecture named by the training method.

812 When method == 'learned_options' is true, enter this block; otherwise continue to the next case.

813 Return LearnedOptionActorCritic(env) to the caller; this ends the function and supplies its result.

814 Return the value produced by this expression to the caller.

11.3. Function `load_checkpoint`

Reload one of this package's own checkpoints for independent evaluation.

```
816 def load_checkpoint(path):
817     """Reload one of this package's own checkpoints for independent evaluation."""
818     checkpoint = torch.load(path, map_location='cpu', weights_only=True)
819     if checkpoint.get('observation_version') != OBSERVATION_VERSION:
820         raise ValueError('Checkpoint uses an older observation layout. Retrain with
        ↪  efficiency/temperature inputs.')
821     config = Config(**checkpoint['config'])
822     model = make_model(FleetMaintenanceEnv(config), checkpoint['method'])
823     model.load_state_dict(checkpoint['state_dict'])
824     model.eval()
825     return (config, checkpoint['method'], model)
```

Line-by-line explanation.

816 Define load_checkpoint with the shown parameters/defaults; its purpose is to rebuild a compatible model and load saved parameters for evaluation.

817 This docstring states load_checkpoint's purpose/contract: Reload one of this package's own checkpoints for independent evaluation. It is documentation text, not an executable operation.

818 Store the result in checkpoint so later lines can use it.

819 When checkpoint.get('observation_version') != OBSERVATION_VERSION is true, enter this block; otherwise continue to the next case.

820 Raise ValueError('Checkpoint uses an older observation layout. Retrain with efficiency/temperature inputs.'); invalid input stops here instead of creating inconsistent state.

821 Store the result in config so later lines can use it.

822 Store the result in model so later lines can use it.

823 Call model.load_state_dict to perform this step of the surrounding algorithm.

824 Call model.eval to perform this step of the surrounding algorithm.

825 Return these values together; callers unpack them in the order shown.

11.4. Function smdp_gae

train calls this after collecting a rollout. For a decision lasting d days, it uses gamma raised to d for the next-value bootstrap and (gamma times lambda) raised to d for advantage propagation. Terminal transitions set the bootstrap multiplier to zero.

```
827 def smdp_gae(rewards, values, next_values, dones, durations, gamma=0.99, lam=0.95):
828     """Backward GAE with gamma**duration and lambda**duration per decision.
829
830     rewards already contain the discounted primitive rewards inside each option.
831     A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.
832     """
833     advantage = np.zeros(len(rewards), dtype=np.float32)
834     carry = 0.0
835     for t in range(len(rewards) - 1, -1, -1):
836         live = 1.0 - float(dones[t])
837         delta = rewards[t] + gamma ** durations[t] * next_values[t] * live - values[t]
838         carry = delta + (gamma * lam) ** durations[t] * live * carry
839         advantage[t] = carry
840     return (advantage, advantage + np.asarray(values))
```

Line-by-line explanation.

827 Define smdp_gae with the shown parameters/defaults; its purpose is to compute generalized advantages when each manager decision can span multiple primitive days.

828 This docstring states smdp_gae's purpose/contract: Backward GAE with gamma**duration and lambda**duration per decision. It is documentation text, not an executable operation.

830 This docstring line states: rewards already contain the discounted primitive rewards inside each option.

831 This docstring line states: A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.

832 This closes the documentation string. Python stores the preceding text as documentation rather than executing it.

833 Initialize advantage with zeros as a neutral container for the GAE estimate used by PPO; later lines fill or update it.

834 Compute the later-decision advantage propagated backward and store it in carry; later checks/updates use this named intermediate.

835 Loop through range(len(rewards) - 1, -1, -1), handling each t once.

836 Store the result in live so later lines can use it.

837 TD residual = option reward + next value discounted across all primitive days - current value. live is zero at terminal boundaries, so no terminal bootstrap is added.

838 Build backward GAE; discount and lambda are each adjusted for the option's duration, and live prevents advantages crossing episode boundaries.

839 Compute the GAE estimate used by PPO and store it in advantage[t]; later checks/updates use this named intermediate.

840 Return these values together; callers unpack them in the order shown.

11.5. Function train

This is the learning loop. It samples from model.act(), asks the simulator or option executor for the resulting transition, computes advantages and targets, and updates model parameters. It never updates FleetMaintenanceEnv's rules or Config values.

```
def train(config, method='conditional', steps=4096, seed=0, rollout_size=128):
    """Interaction budget counts actual simulated days, not manager decisions."""
    if steps < 1 or rollout_size < 2:
        raise ValueError('positive budget and rollout>=2 required')
    torch.set_num_threads(1)
    torch.manual_seed(seed)
    env = FleetMaintenanceEnv(config)
    model = make_model(env, method)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.0003)
    rng = np.random.default_rng(seed)
    episode = 0
    obs, _ = env.reset(seed=seed)
    simulated = 0
    logs = []
    while simulated < steps:
        data = {k: [] for k in ('obs', 'action', 'logp', 'value', 'next_value', 'reward', 'done',
        ↪  'duration')}
        while len(data['obs']) < rollout_size and simulated < steps:
            with torch.no_grad():
                action, logp, _, value = model.act(torch.tensor(obs))
            chosen = action[0].numpy()
            if method in ('options', 'learned_options'):
                macro = int(chosen) if method == 'options' else int(chosen[0])
                plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
                next_obs, reward, done, truncated, info, duration, _ = execute_option(env, macro, obs,
                ↪  max_days=min(14, steps - simulated), planned_action=plan)
            else:
                next_obs, reward, done, truncated, info = env.step(chosen)
                duration = 1
            terminal = done or truncated
            with torch.no_grad():
                nv = float(model.value(torch.tensor(next_obs).unsqueeze(0))[0])
            for key, item in [('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp',
            ↪  float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward),
            ↪  ('done', terminal), ('duration', duration)]:
                data[key].append(item)
            simulated += duration
            obs = next_obs
            if terminal:
                logs.append({'episode': episode, 'simulated_days': simulated, **info['metrics']})
                episode += 1
                obs, _ = env.reset(seed=seed + episode)
        advantages, targets = smdp_gae(data['reward'], data['value'], data['next_value'], data['done'],
        ↪  data['duration'])
        tensors = {k: torch.tensor(np.asarray(v)) for k, v in data.items() if k in ('obs', 'action',
        ↪  'logp')}
        advantages = torch.tensor(advantages)
        if len(advantages) > 1:
            advantages = (advantages - advantages.mean()) / (advantages.std(unbiased=False) + 1e-08)
        targets = torch.tensor(targets, dtype=torch.float32)
        for _ in range(4):
            ids = rng.permutation(len(targets))
            for start in range(0, len(ids), 64):
                idx = ids[start:start + 64]
                _, newlog, entropy, values = model.act(tensors['obs'][idx], tensors['action'][idx])
                ratio = torch.exp(newlog - tensors['logp'][idx])
                surrogate = torch.minimum(ratio * advantages[idx], ratio.clamp(0.8, 1.2) *
                ↪  advantages[idx])
                loss = -surrogate.mean() + 0.5 * ((values - targets[idx]) ** 2).mean() - 0.001 *
                ↪  entropy.mean()
                if not torch.isfinite(loss):
                    raise RuntimeError('nonfinite PPO loss')
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5)
                optimizer.step()
```

```
900     return (model, logs, simulated)
```

Line-by-line explanation.

842 Define train with the shown parameters/defaults; its purpose is to collect rollouts, compute advantages and targets, then optimize the policy with PPO.

843 This docstring states train's purpose/contract: Interaction budget counts actual simulated days, not manager decisions. It is documentation text, not an executable operation.

844 When steps < 1 or rollout_size < 2 is true, enter this block; otherwise continue to the next case.

845 Raise ValueError('positive budget and rollout>=2 required'); invalid input stops here instead of creating inconsistent state.

846 Limit PyTorch to one CPU thread for reproducible, predictable training runs.

847 Seed PyTorch's random generator so policy initialization and sampling can be reproduced.

848 Store the result in env so later lines can use it.

849 Store the result in model so later lines can use it.

850 Store the result in optimizer so later lines can use it.

851 Store the result in rng so later lines can use it.

852 Compute the current training episode index and store it in episode; later checks/updates use this named intermediate.

853 Unpack the observation and info from reset(); assign info to _ because this caller does not use it.

854 Store the result in simulated so later lines can use it.

855 Store the result in logs so later lines can use it.

856 When simulated < steps is true, enter this block; otherwise continue to the next case.

857 Store the result in data so later lines can use it.

858 When len(data['obs']) < rollout_size and simulated < steps is true, enter this block; otherwise continue to the next case.

859 Disable gradient tracking in this block. Rollout and evaluation only read the model, so they do not need a backward graph.

860 Receive the action, log-probability, entropy, and value. The underscores discard outputs this call does not need.

861 Store the result in chosen so later lines can use it.

862 When method in ('options', 'learned_options') is true, enter this block; otherwise continue to the next case.

863 Compute the manager's high-level option ID and store it in macro; later checks/updates use this named intermediate.

864 Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

865 Unpack the return values from execute_option(env, macro, obs, max_days=min(14, steps - simulated), planned_action=plan) into next_obs, reward, done, truncated, info, duration; underscore entries are intentionally ignored.

866 Otherwise, follow this alternative branch.

867 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

868 Compute the primitive days consumed by a job or option and store it in duration; later checks/updates use this named intermediate.

869 Store the result in terminal so later lines can use it.

870 Disable gradient tracking in this block. Rollout and evaluation only read the model, so they do not need a backward graph.

871 Store the result in nv so later lines can use it.

872 Loop through [('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp', float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward), ('done', terminal), ('duration', duration)], handling each key, item once.

873 Add this item to data[key] so it stays available for later use.

874 Add duration to the running total in simulated.

875 Update obs to the current encoded state before the next decision.

876 When terminal is true, enter this block; otherwise continue to the next case.

877 Add this item to logs so it stays available for later use.

878 Add 1 to the running total in episode.

879 Unpack the observation and info from reset(); assign info to _ because this caller does not use it.

880 Separate this result into advantages, targets for the steps that follow.

881 Store the result in tensors so later lines can use it.

882 Store the result in advantages so later lines can use it.

883 When len(advantages) > 1 is true, enter this block; otherwise continue to the next case.

884 Store the result in advantages so later lines can use it.

885 Store the result in targets so later lines can use it.

886 Loop through range(4), handling each _ once.

887 Store the result in ids so later lines can use it.

888 Loop through range(0, len(ids), 64), handling each start once.

889 Store the result in idx so later lines can use it.

890 Receive the action, log-probability, entropy, and value. The underscores discard outputs this call does not need.

891 Exponentiate the log-probability difference to obtain πnew(a|s)/πold(a|s), PPO's policy-update ratio.

892 Take the smaller unclipped/clipped objective; clipping the ratio to 0.8–1.2 limits how far this minibatch moves the policy likelihood.

893 Combine policy loss, half-weighted squared critic error, and a small negative entropy term that rewards exploration.

894 When not torch.isfinite(loss) is true, enter this block; otherwise continue to the next case.

895 Raise RuntimeError('nonfinite PPO loss'); invalid input stops here instead of creating inconsistent state.

896 Clear gradients from the previous minibatch because PyTorch accumulates gradients by default.

897 Differentiate this minibatch loss with respect to model parameters and populate their gradients.

898 Rescale gradients if their combined norm exceeds 0.5, limiting unstable parameter updates.

899 Apply one Adam parameter update using the gradients just computed.

900 Return these values together; callers unpack them in the order shown.

11.6. Function evaluate

This runs separate episodes with selected seeds. Learned policies use deterministic action selection; the threshold method bypasses the neural model. The function records outcomes and does not call the optimizer.

```
902 def evaluate(config, method, model=None, seeds=range(10000, 10010)):
903     records = []
904     for seed in seeds:
905         env = FleetMaintenanceEnv(config)
906         obs, _ = env.reset(seed=int(seed))
907         decisions = 0
908         while True:
909             if method == 'threshold':
910                 obs, _, done, _, info = env.step(threshold_action(env, obs))
911             else:
912                 with torch.no_grad():
913                     action, _, _, _ = model.act(torch.tensor(obs), deterministic=True)
914                 if method in ('options', 'learned_options'):
915                     chosen = action[0].numpy()
916                     macro = int(chosen) if method == 'options' else int(chosen[0])
917                     plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
918                     obs, _, done, _, info, _, _ = execute_option(env, macro, obs, planned_action=plan)
919                 else:
920                     obs, _, done, _, info = env.step(action[0].numpy())
921             decisions += 1
922             if done:
923                 break
924         records.append({'seed': int(seed), 'decisions': decisions, 'availability':
        ↪  info['metrics']['flights'] / (config.aircraft * config.horizon), **info['metrics']})
925     return records
```

Line-by-line explanation.

902 Define evaluate with the shown parameters/defaults; its purpose is to run fixed-seed episodes and record simulator performance metrics.

903 Store the result in records so later lines can use it.

904 Loop through seeds, handling each seed once.

905 Store the result in env so later lines can use it.

906 Unpack the observation and info from reset(); assign info to _ because this caller does not use it.

907 Store the result in decisions so later lines can use it.

908 Continue the loop until a stopping condition triggers break or return.

909 When method == 'threshold' is true, enter this block; otherwise continue to the next case.

910 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

911 Otherwise, follow this alternative branch.

912 Disable gradient tracking in this block. Rollout and evaluation only read the model, so they do not need a backward graph.

913 Receive the action, log-probability, entropy, and value. The underscores discard outputs this call does not need.

914 When method in ('options', 'learned_options') is true, enter this block; otherwise continue to the next case.

915 Store the result in chosen so later lines can use it.

916 Compute the manager's high-level option ID and store it in macro; later checks/updates use this named intermediate.

917 Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

918 Unpack the return values from execute_option(env, macro, obs, planned_action=plan) into obs, done, info; underscore entries are intentionally ignored.

919 Otherwise, follow this alternative branch.

920 Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

921 Add 1 to the running total in decisions.

922 Check the episode/option stop condition done; when true, stop collecting transitions for this segment.

923 Stop this loop because its exit condition is satisfied.

924 Add this item to records so it stays available for later use.

925 Return records to the caller; this ends the function and supplies its result.

11.7. Function `main`

This is the command-line entry point. It selects configurations and methods, invokes train or checkpoint evaluation, and persists summaries and model state.

```
927 def main():
928     parser = argparse.ArgumentParser()
929     parser.add_argument('--method', choices=['flat', 'conditional', 'shared', 'options',
        ↪  'learned_options', 'all'], default='all')
```

```
930     parser.add_argument('--steps', type=int, default=4096)
931     parser.add_argument('--aircraft', type=int, default=7)
932     parser.add_argument('--horizon', type=int, default=90)
933     parser.add_argument('--seed', type=int, default=0)
934     parser.add_argument('--eval-episodes', type=int, default=10)
935     parser.add_argument('--output', default='runs/smoke')
936     args = parser.parse_args()
937     config = Config(aircraft=args.aircraft, horizon=args.horizon)
938     destination = Path(args.output)
939     destination.mkdir(parents=True, exist_ok=True)
940     result = {'config': asdict(config), 'training_seed': args.seed, 'requested_steps': args.steps,
        ↪  'observation_version': OBSERVATION_VERSION, 'status': 'preliminary teaching experiment; not a
        ↪  paper reproduction or convergence claim', 'comparison_note': 'options changes manager decision
        ↪  frequency; interpret as timing + representation', 'evaluation': {'threshold': evaluate(config,
        ↪  'threshold', seeds=range(10000, 10000 + args.eval_episodes))}}
941     methods = ['flat', 'conditional', 'shared', 'options', 'learned_options'] if args.method == 'all'
        ↪  else [args.method]
942     for method in methods:
943         model, logs, days = train(config, method, args.steps, args.seed)
944         torch.save({'state_dict': model.state_dict(), 'config': asdict(config), 'method': method,
            ↪  'training_seed': args.seed, 'simulated_days': days, 'observation_version':
            ↪  OBSERVATION_VERSION}, destination / f'{method}.pt')
945         result['evaluation'][method] = evaluate(config, method, model, seeds=range(10000, 10000 +
            ↪  args.eval_episodes))
946         result[method + '_training'] = logs
947         print(method, 'days', days, 'mean test return', np.mean([r['return'] for r in
            ↪  result['evaluation'][method]]), flush=True)
948     (destination / 'results.json').write_text(json.dumps(result, indent=2), encoding='utf-8')
949     summary = ['# Preliminary comparison', '', result['status'], '', result['comparison_note'], '', '|
        ↪  Method | Return | Availability | Failed flights | Rejected plans | Decisions |',
        ↪  '|---|---:|---:|---:|---:|---:|']
950     for method, records in result['evaluation'].items():
951         mean = lambda k: np.mean([r[k] for r in records])
952         summary.append(f"| {method} | {mean('return'):.2f} | {mean('availability'):.3f} |
            ↪  {mean('flight_failures'):.1f} | {mean('rejections'):.1f} | {mean('decisions'):.1f} |")
953     (destination / 'summary.md').write_text('\n'.join(summary) + '\n', encoding='utf-8')
954     print('Saved', destination.resolve())
```

Line-by-line explanation.

927 Define main with the shown parameters/defaults; its purpose is to parse command-line settings, train/evaluate methods, and write results.

928 Store the result in parser so later lines can use it.

929 Separate this result into parser.add_argument('–method', choices for the steps that follow.

930 Separate this result into parser.add_argument('–steps', type for the steps that follow.

931 Separate this result into parser.add_argument('–aircraft', type for the steps that follow.

932 Separate this result into parser.add_argument('–horizon', type for the steps that follow.

933 Separate this result into parser.add_argument('–seed', type for the steps that follow.

934 Separate this result into parser.add_argument('–eval-episodes', type for the steps that follow.

935 Separate this result into parser.add_argument('–output', default for the steps that follow.

936 Store the result in args so later lines can use it.

937 Store the result in config so later lines can use it.

938 Store the result in destination so later lines can use it.

939 Store the result in destination.mkdir(parents so later lines can use it.

940 Store the result in result so later lines can use it.

941 Store the result in methods so later lines can use it.

942 Loop through methods, handling each method once.

943 Separate this result into model, logs, days for the steps that follow.

944 Save the trained model parameters and experiment metadata as a checkpoint for this method.

945 Store the result in result['evaluation'][method] so later lines can use it.

946 Store the result in result[method + '_training'] so later lines can use it.

947 Print this method's simulated-day count and mean test return as progress information.

948 Serialize the results as JSON and save them in the output directory.

949 Store the result in summary so later lines can use it.

950 Loop through result['evaluation'].items(), handling each method, records once.

951 Store the result in mean so later lines can use it.

952 Add this item to summary so it stays available for later use.

953 Write the comparison summary as a Markdown file in the output directory.

954 Show progress/result information to the person running the script.

11.8. Module-level declarations

This block establishes the module's imports, constants, or public interface.

```
955
956
957 OPERATE_UNTIL_LOW = 0
958 REPLACE_NOW = 1
959
960
961
```

Line-by-line explanation.

957 Define OPERATE_UNTIL_LOW as a shared modeling constant: the option ID for operation until health is low.

958 Define REPLACE_NOW as a shared modeling constant: the option ID for immediate replacement.

12. Interpretation and limitations

The teaching pipeline separates the policy from the simulator. The policy selects actions from the observation available to it; the simulator decides how those actions change hidden health, maintenance status, and fleet outcomes. Training updates the policy and critic from

simulated experience. Evaluation measures behavior under the chosen synthetic assumptions. It does not establish real-world engine reliability, calibrated hazard probabilities, or deployment safety.

The learned hierarchy also changes the decision schedule. A manager action can span several simulated days, while a daily policy acts at every step. The duration-aware return addresses that difference in the update, but a comparison between these methods still changes both action structure and decision timing. The report therefore treats the options results as comparisons within this environment, not as a controlled isolation of hierarchy alone.

The code is best read as a tutorial implementation: the small environments teach state, action, reward, and API structure; the fleet environment supplies a synthetic maintenance problem; the actor-critic classes define what is learned; the trainer connects sampled decisions to parameter updates; and evaluation records outcomes without learning. Related studies in the supplied literature review address selective maintenance and aircraft-fleet scheduling under partial observability, but their models and assumptions should not be conflated with this tutorial simulator [6, 7].

References

[1] Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. Second edition. MIT Press. http://incompleteideas.net/book/the-book-2nd.html

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint*, arXiv:1707.06347. https://arxiv.org/abs/1707.06347

[3] Stable-Baselines3 contributors. (n.d.). Custom policy networks. *Stable-Baselines3 documentation*. https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html

[4] Sutton, R. S., Precup, D., and Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2), 181–211. https://doi.org/10.1016/S0004-3702(99)00052-1

[5] Bacon, P.-L., Harb, J., and Precup, D. (2017). The option-critic architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1). https://doi.org/10.1609/aaai.v31i1.10916

[6] Liu, Y., Chen, Y., and Jiang, T. (2020). Dynamic selective maintenance optimization for multi-state systems over a finite horizon: A deep reinforcement learning approach. *European Journal of Operational Research*, 283, 166–181. https://doi.org/10.1016/j.ejor.2019.10.049

[7] Tseremoglou, I., and Santos, B. F. (2024). Condition-based maintenance scheduling of an aircraft fleet under partial observability: A deep reinforcement learning approach. *Reliability Engineering & System Safety*, 241, 109582. https://doi.org/10.1016/j.ress.2023.109582